Inteligência Computacional

Redes Neurais

Prof. Aldebaro Klautau
Universidade Federal do Pará (UFPA)

www.lasse.ufpa.br & www.ppgee.ufpa.br

Tensorflow (Cap. 9)

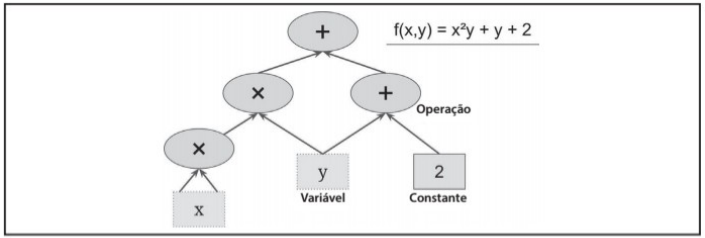


Figura 9-1. Um grafo de cálculo simples

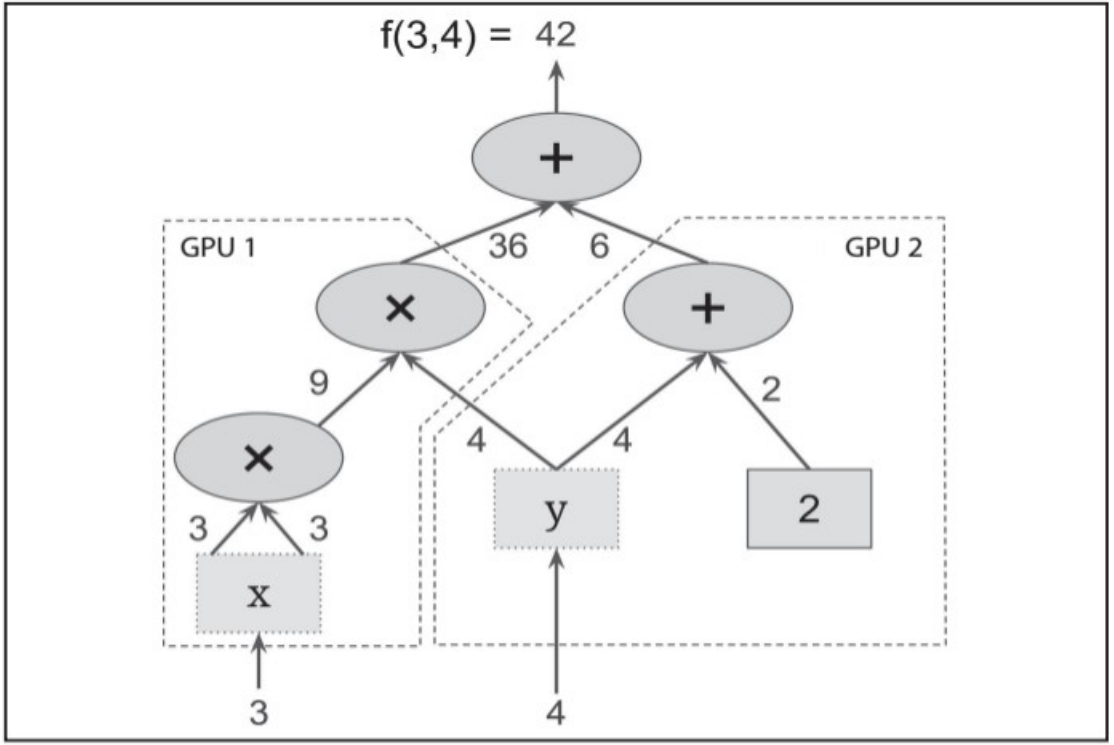
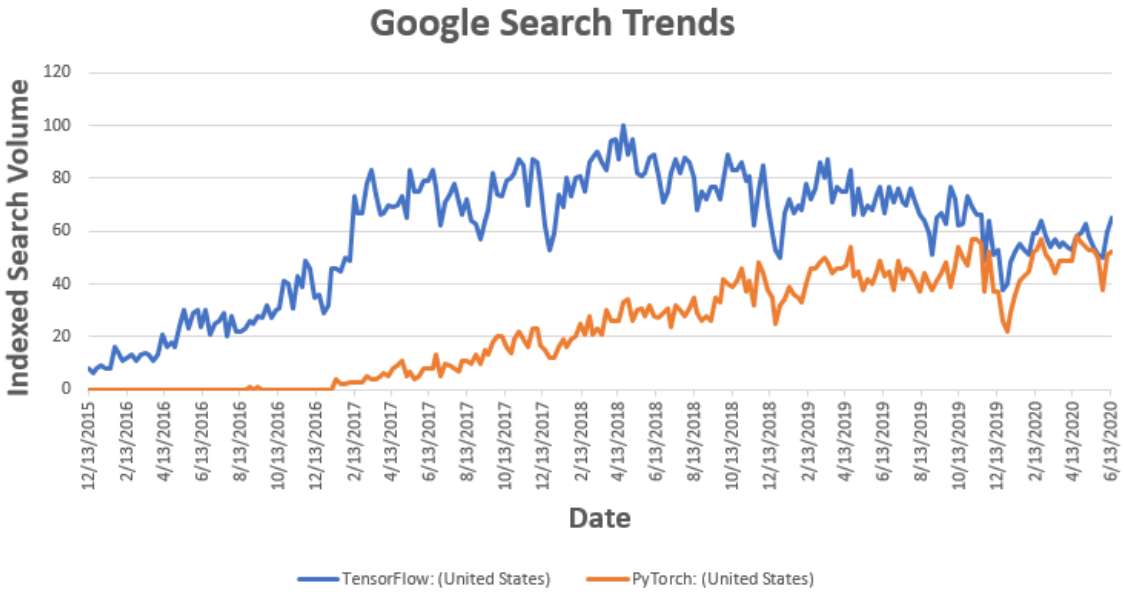


Figura 9-2. Computação paralela em várias CPUs/GPUs/servidores

Bibliotecas: autodiff, etc.

Tabela 9-1. Bibliotecas de Aprendizado Profundo de código aberto (não é uma lista completa)

Biblioteca	API	Plataformas	Iniciado por	Ano
Caffe	Python, C++, Matlab	Linux, macOS, Windows	Y. Jia, UC Berkeley (BVLC)	2013
Deeplearning4j	Java, Scala, Clojure	Linux, macOS, Windows, Android	A. Gibson, J.Patterson	2014
H2O	Python, R	Linux, macOS, Windows	H2O.ai	2014
MXNet	Python, C++, others	Linux, macOS, Windows, iOS, Android	DMLC	2015
TensorFlow	Python, C++	Linux, macOS, Windows, iOS, Android	Google	2015
Theano	Python	Linux, macOS, iOS	Universidade de Montreal	2010
Torch	C++, Lua	Linux, macOS, iOS, Android	R. Collobert, K. Kavukcuoglu, C. Farabet	2002



Perceptron e limitação no OU exclusivo (XOR)

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.linear_model import Perceptron

iris = load_iris()
X = iris.data[:, (2, 3)]  # comprimento da pétala, largura da pétala
y = (iris.target == 0).astype(np.int)  # Iris Setosa?

per_clf = Perceptron(random_state=42)
per_clf.fit(X, y)

y_pred = per_clf.predict([[2, 0.5]])
```

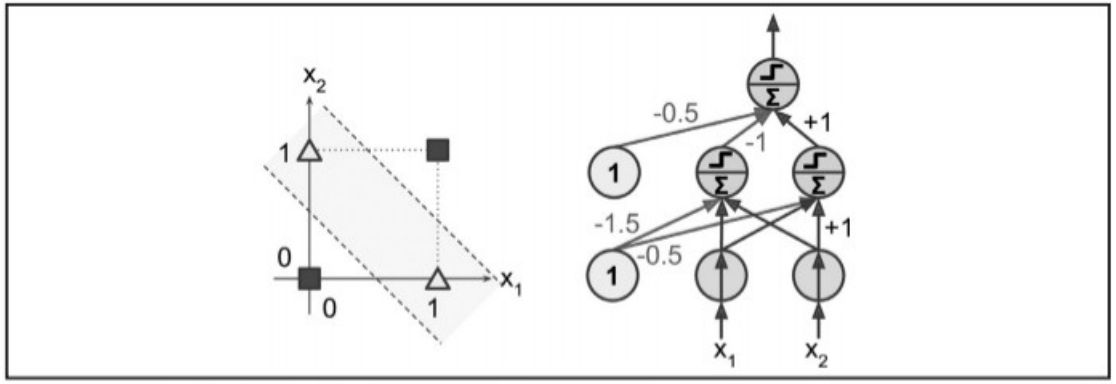
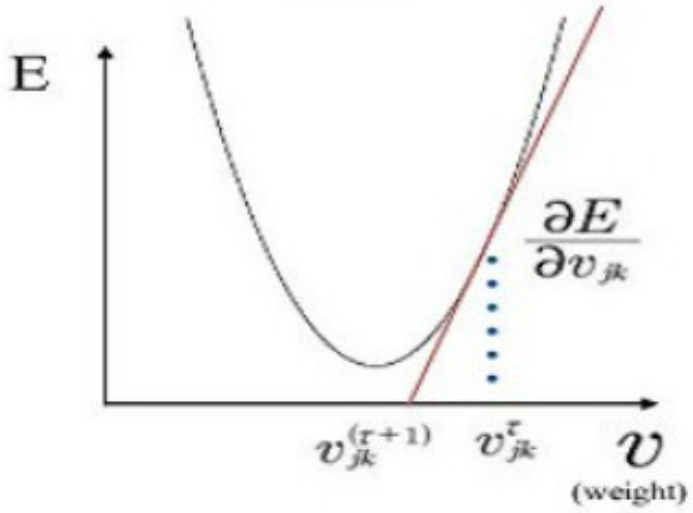


Figura 10-6. Problema de classificação XOR e um MLP que o soluciona

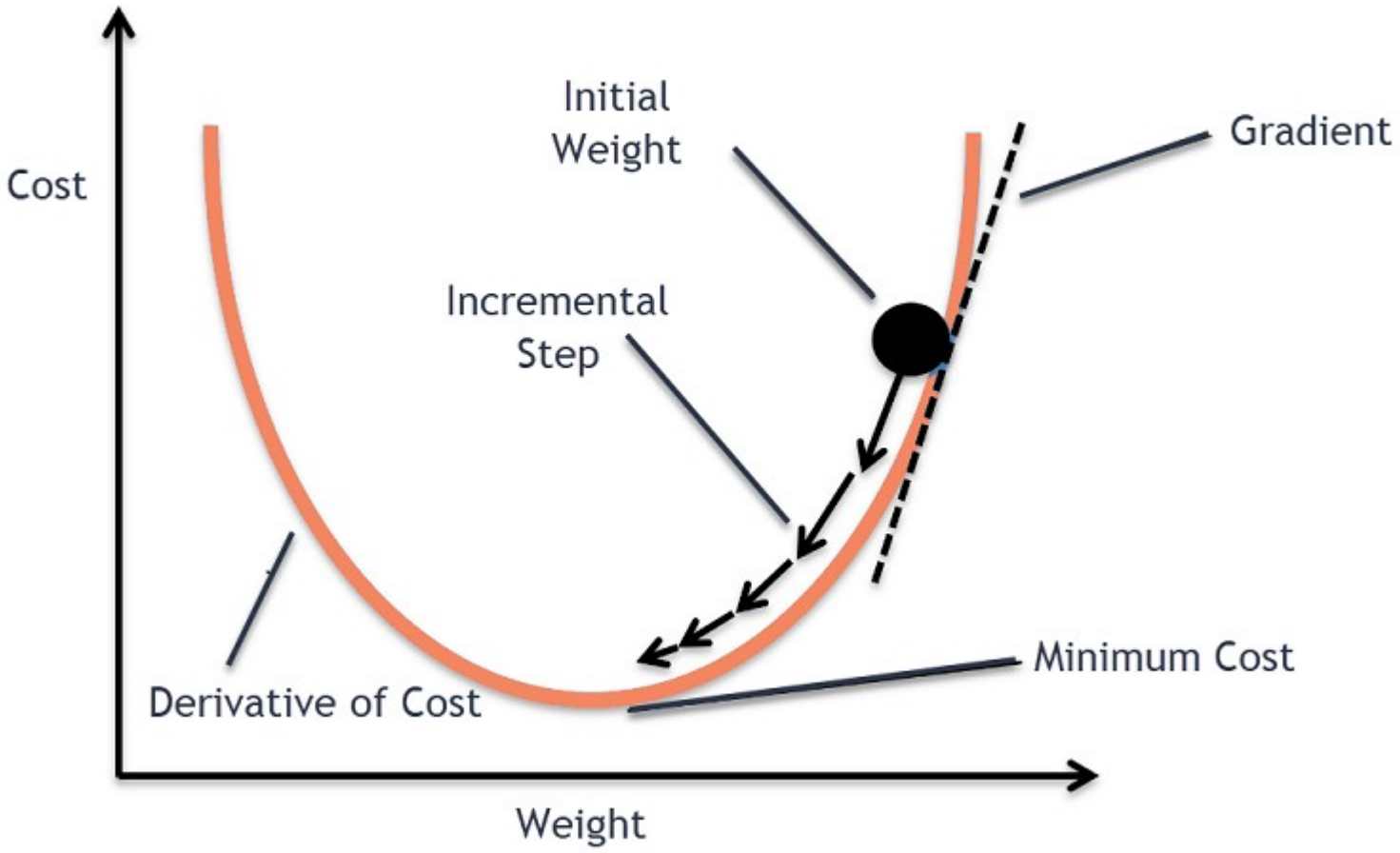
Delta rule or Widrow & Hoff Learning rule or the Least Mean Square (LMS) rule (de 1960)



$$v_{jk}^{(\tau+1)} = v_{jk}^{\tau} + \Delta v_{jk}$$

$$\Delta v_{jk} = -\eta \frac{\partial E}{\partial v_{jk}}$$

$v_{jk}^{(\tau+1)}$	new weight
v_{jk}^{τ}	current weight
η	learning rate
E	Error Function


Initial
Weight
Gradient
Cost
Incremental
Step
Minimum Cost
Derivative of Cost
Weight

Perceptron com múltiplas camadas

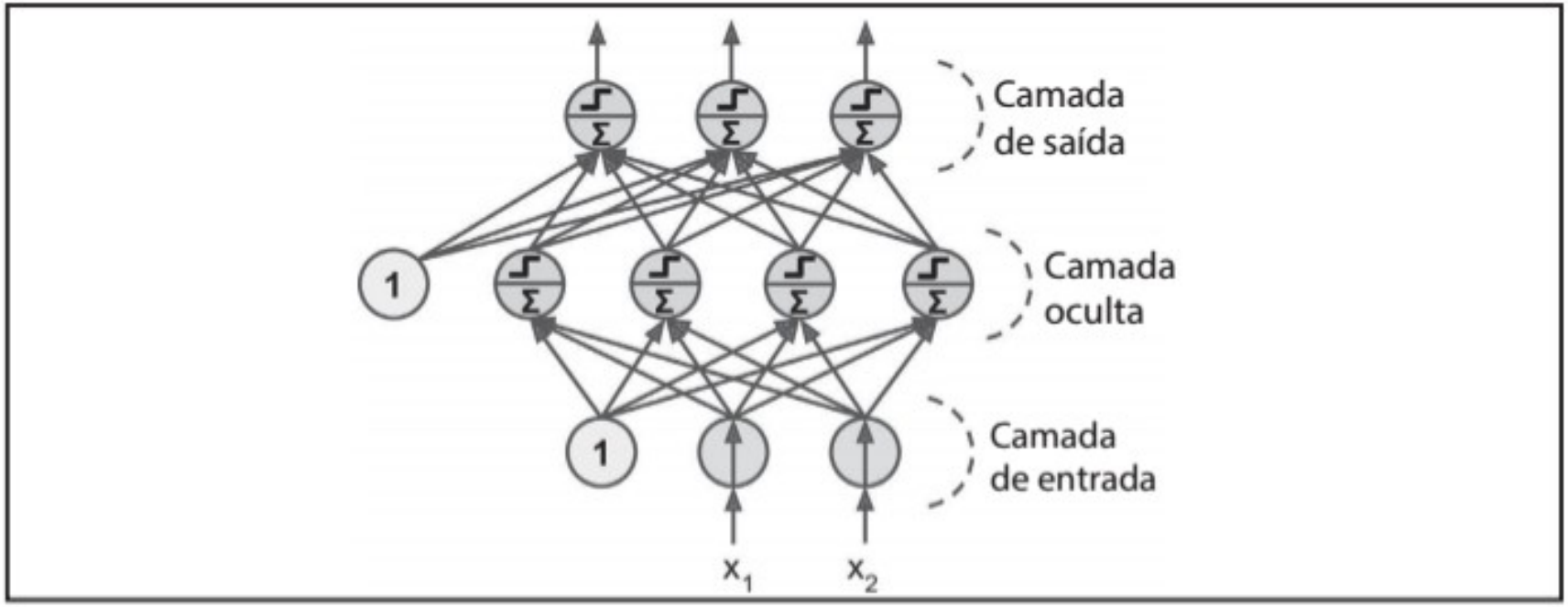
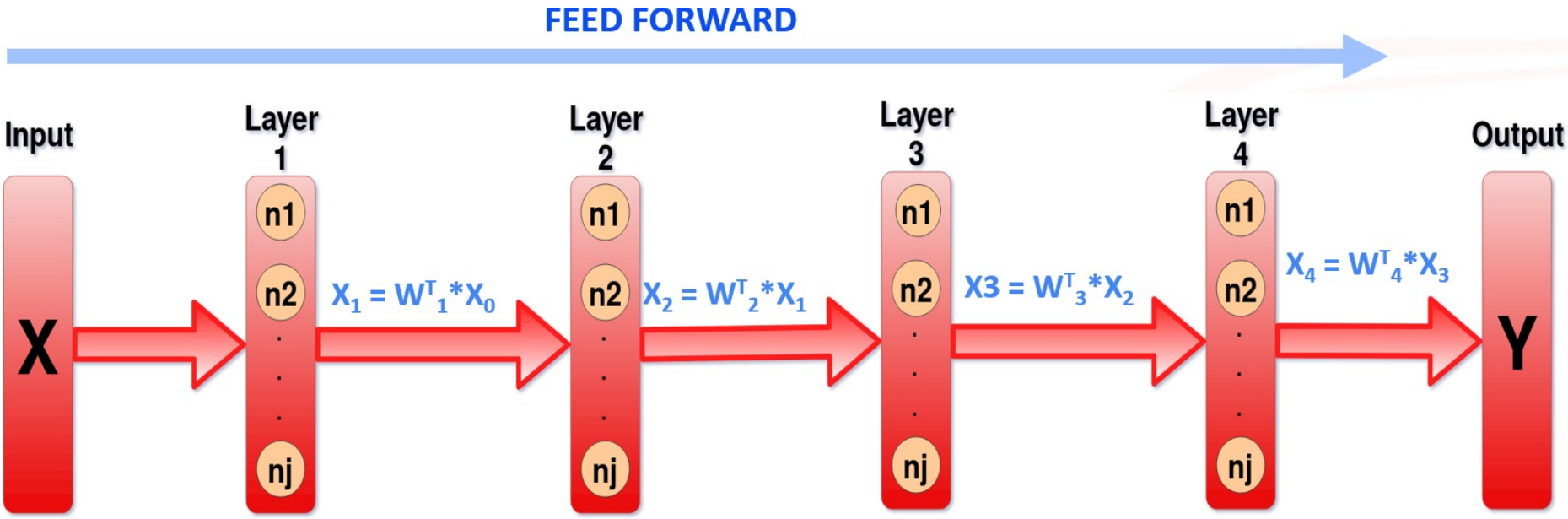


Figura 10-7. Multicamada Perceptron

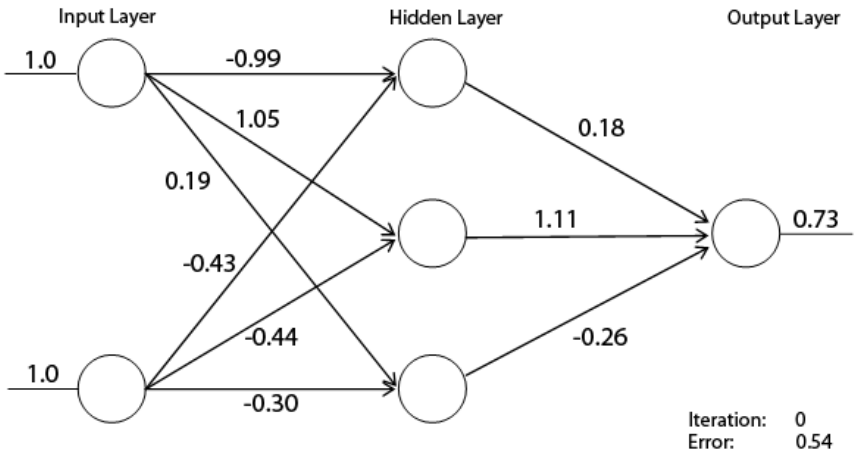
Notação matricial



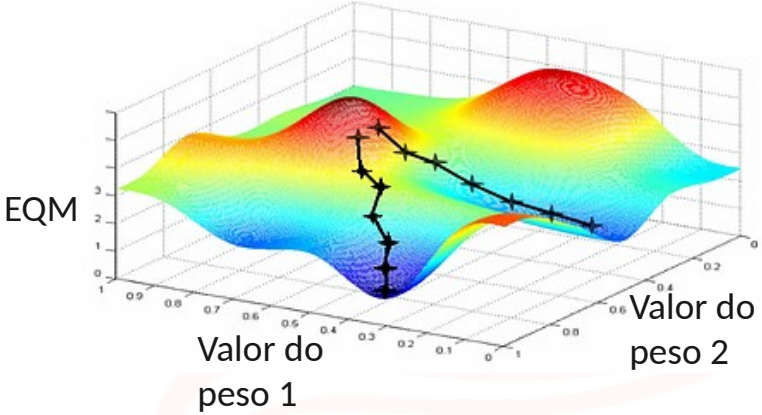
A função não-linear, de ativação, não está indicada acima

Estado-da-arte no treinamento de rede neural

Tensorflow (Google), PyTorch (Facebook) e outros “frameworks” implementam algoritmos sofisticados que usam “backpropagation”, diferenciação automática e gradiente estocástico descendente (SGD)



Problemas: “underflow” ou “overflow”, do gradiente, mínimos locais, “overfitting”



Humanidade levou **20 anos** para aprender a treinar redes profundas com milhões de neurônios

Funções de ativação

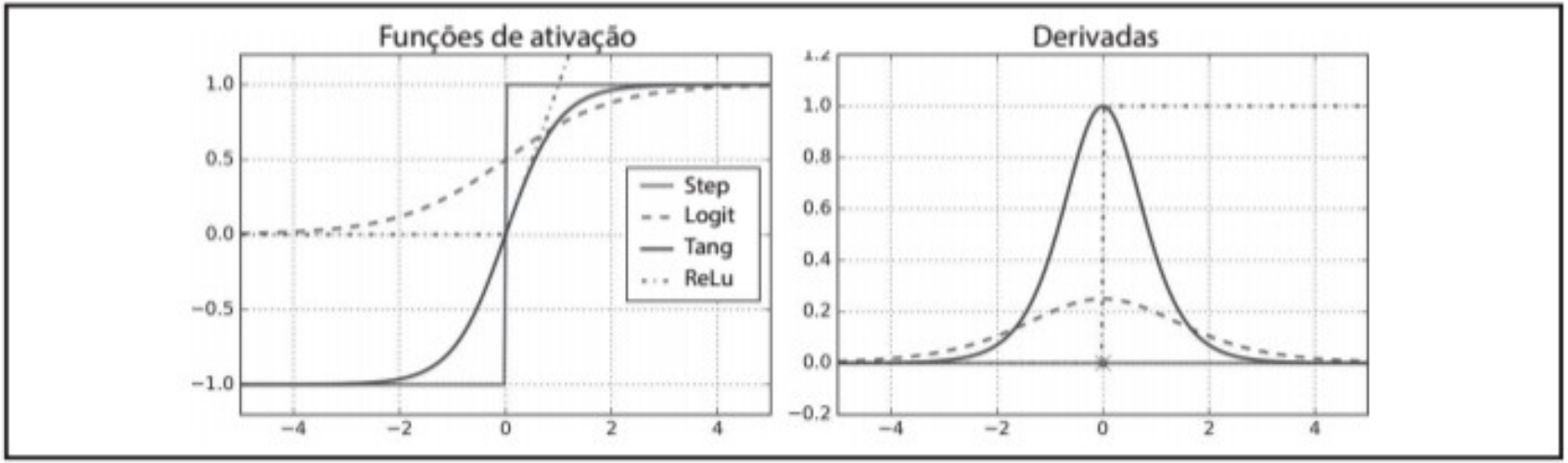
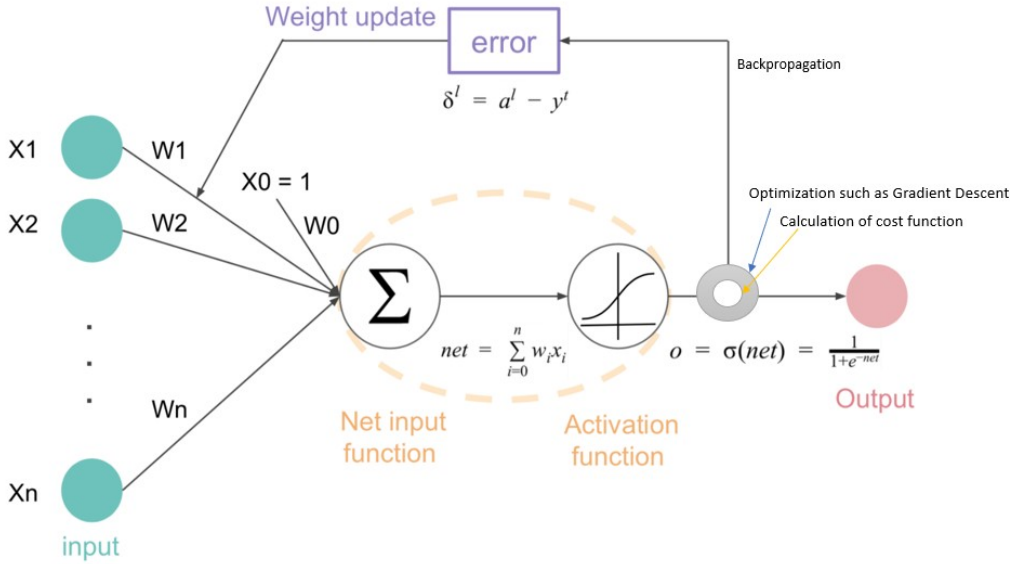


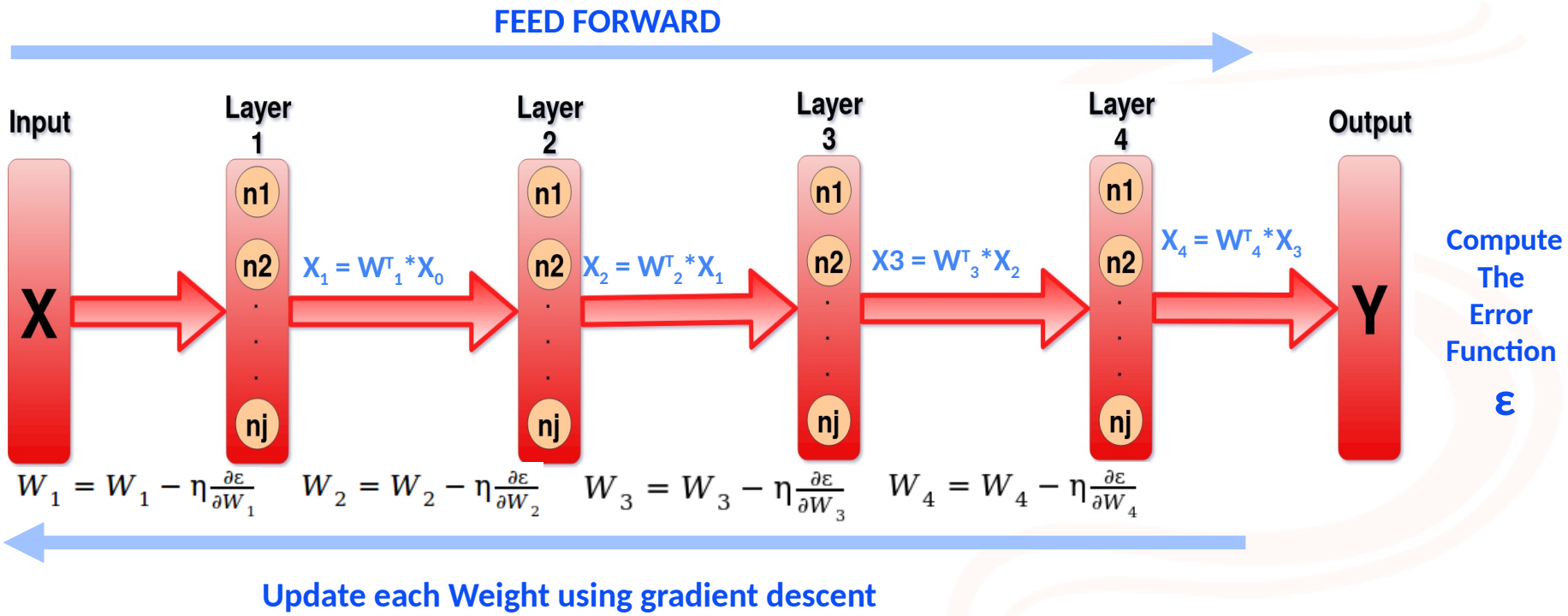
Figura 10-8. Funções de Ativação e suas derivadas

Queremos distinguir: Backpropagation de gradient descent

- GD: derivative of a function gives the direction in which the function increases, and its negative, the direction in which the function decreases
- How to train inner layers? Backprop:



Vanishing / Exploding Gradient (picture)



From: https://web.stanford.edu/class/cs221/lectures/learningXX.pdf

Vanishing/Exploding Gradients Problems

- deep neural networks suffer from unstable gradients; different layers may learn at widely different speeds. In 2010: “Understanding the Difficulty of Training Deep Feedforward Neural Networks” by X. Glorot and Y. Bengio

$$W_4 = W_4 - \eta \frac{\partial \varepsilon}{\partial W_4}$$

$$W_3 = W_3 - \eta \frac{\partial \varepsilon}{\partial W_4} \frac{\partial W_4}{\partial W_3}$$

$$W_2 = W_2 - \eta \frac{\partial \varepsilon}{\partial W_4} \frac{\partial W_4}{\partial W_3} \frac{\partial W_3}{\partial W_2}$$

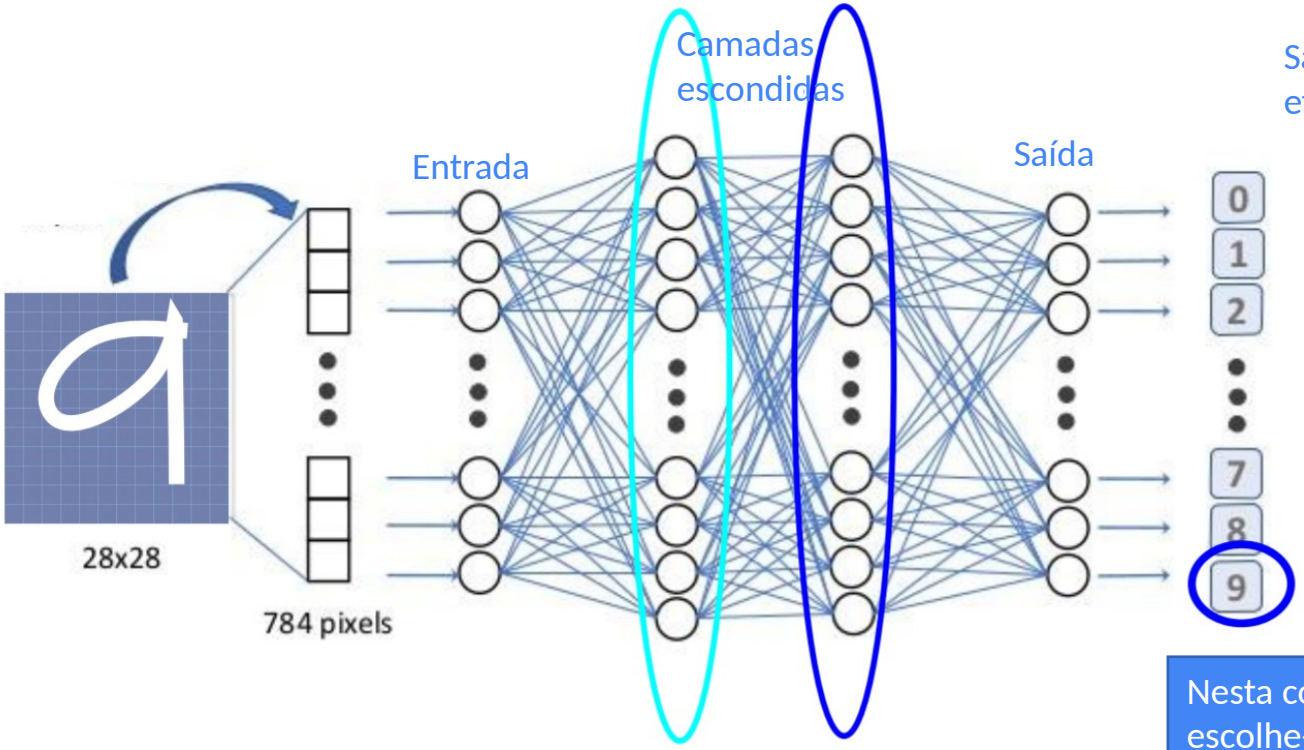
$$W_1 = W_1 - \eta \frac{\partial \varepsilon}{\partial W_4} \frac{\partial W_4}{\partial W_3} \frac{\partial W_3}{\partial W_2} \frac{\partial W_2}{\partial W_1}$$

The number of multiplications in the chain rule increases as the algorithm proceeds.

Codificação da saída da rede neural

Cada conexão (seta) corresponde a um parâmetro ou peso que a rede precisa aprender

Exemplo: detecção dos 10 algarismos (dígitos) usando codificação “one-hot”



Saída durante etapa de teste	Saída desejada em “one-hot encoding” (rótulo ou “label”)
3.1	0
0.2	0
-0.01	0
...	...
1.5	0
0.8	0
7.3	1

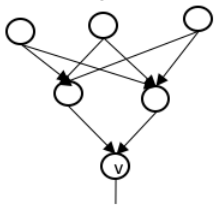
Nesta codificação, escolhe-se o maior valor

https://cedarweb.vsp.ucar.edu/wiki/images/d/d6/Bhatt_DeepLearningStudentTutorial.pdf

Loss functions and function activation

1.Binary Classification

NN model



Layers

input → Dense → Dense → Sigmoid → output

y=0.9

error

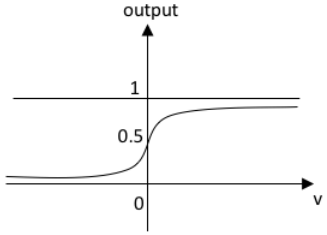
label

t=1.0

Activation

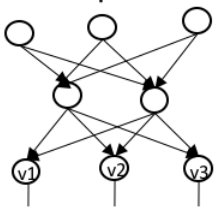
sigmoid function

$$output = \frac{1}{1+e^{-v}}$$



2.Multiclass Classification

NN model



Layers

input → Dense → Dense → Softmax → output

y1=0.2 y2=0.1 y3=0.7

error

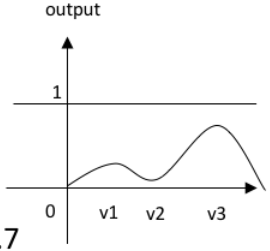
label

t1=0.0 t2=0.0 t3=1.0

Activation

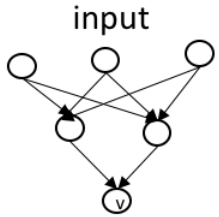
softmax function

$$output_k = \frac{e^k}{\sum_{i=1}^{n} e^i}$$



3.Regression

NN model



Layers

input → Dense → output

y=417

error

label

t=520

Cross Entropy(CE)

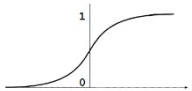
$$L = -\sum t_i log y_i$$

Mean Squared Error(MSE)

$$L = \frac{1}{2}(t-y)^2$$

Output activations and loss functions

Problem	Example of target (p), *C*=4	Example of output (q)	Loss	Output activation
Multiclass classification	[0, 1, 0, 0] (one-hot)	[0.2, 0.4, 0.1, 0.3]	Categorical cross-entropy	Softmax
Multilabel classification	[1, 0, 0, 1]	[0.9, 0.4, 0.7, 0.8]	Binary cross-entropy	Sigmoid
Multivariate regression	[5.2, -3.1, 2]	[5.1, -2, 3.2]	MSE	Linear



Cross-entropy $L(p,q) = H(p) + D_{KL}(p//q)$ indicates how much two distributions differ.
It is applied for each of the *C* elements of output vector in multilabel classification

Typically, increasing level of difficulty: multiclass classification, multilabel classification, multivariate regression (often solved with *C* independent multivariable regressors)

Single lesson:

Representing C classes using $\log_2 C$ bits (binary encoding) is typically outperformed by one-hot encoding

Treinando redes profundas (Cap. 14)

- Ao invés de inicializar os pesos de forma completamente aleatória:

Inicialização Xavier e Inicialização He

Glorot e Bengio propõem em seu artigo uma forma de aliviar significativamente esse problema. Precisamos que o sinal flua corretamente em ambas as direções: na direção *forward* quando fizer previsões e na direção *reverse* ao retropropagar gradientes. Nós não queremos que o sinal desapareça, nem queremos que ele exploda e sature. Para que o sinal flua corretamente, os autores argumentam que a variância das saídas de cada camada deve ser igual à variância de suas entradas,[2] e também que os gradientes tenham igual variância antes e depois de fluir na direção reversa através de uma camada

Tabela 11-1. Parâmetros de inicialização para cada tipo de função de ativação

Função de ativação	Distribuição uniforme [−r, r]	Distribuição normal
Logística	$r = \sqrt{\frac{6}{n_{\text{entradas}} + n_{\text{saídas}}}}$	$\sigma = \sqrt{\frac{2}{n_{\text{entradas}} + n_{\text{saídas}}}}$
Tangente Hiperbólica	$r = 4\sqrt{\frac{6}{n_{\text{entradas}} + n_{\text{saídas}}}}$	$\sigma = 4\sqrt{\frac{2}{n_{\text{entradas}} + n_{\text{saídas}}}}$
ReLU (e suas variantes)	$r = \sqrt{2}\sqrt{\frac{6}{n_{\text{entradas}} + n_{\text{saídas}}}}$	$\sigma = \sqrt{2}\sqrt{\frac{2}{n_{\text{entradas}} + n_{\text{saídas}}}}$

Por padrão, a função `tf.layers.dense()` (introduzida no Capítulo 10) utiliza a inicialização Xavier (com uma distribuição uniforme), mas é possível mudar para a inicialização He usando a função `variance_scaling_initializer()` desta forma:

```
he_init = tf.contrib.layers.variance_scaling_initializer()
hidden1 = tf.layers.dense(X, n_hidden1, activation=tf.nn.relu,
                          kernel_initializer=he_init, name="hidden1")
```

Combate ao "dying ReLUs"

Equação 11-2. Função de ativação ELU

$$\mathrm{ELU}_{\alpha}(z) = \begin{cases} \alpha(\exp(z) - 1) & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases}$$

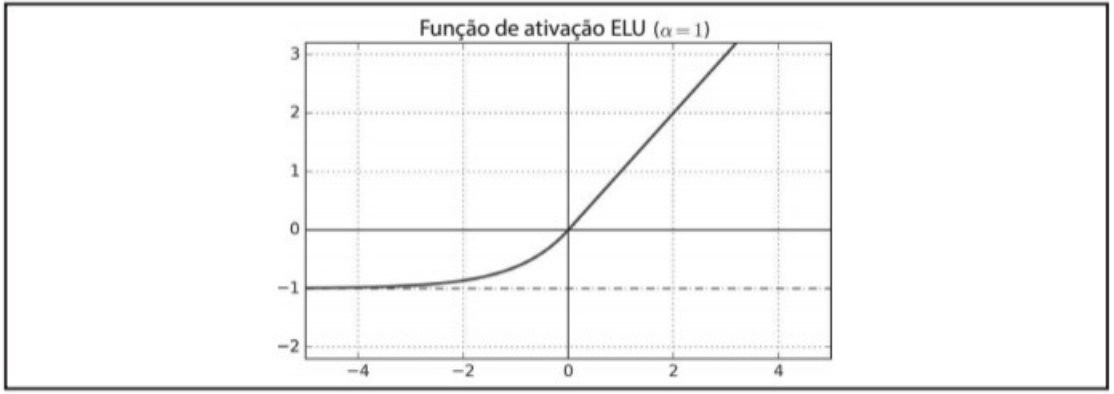
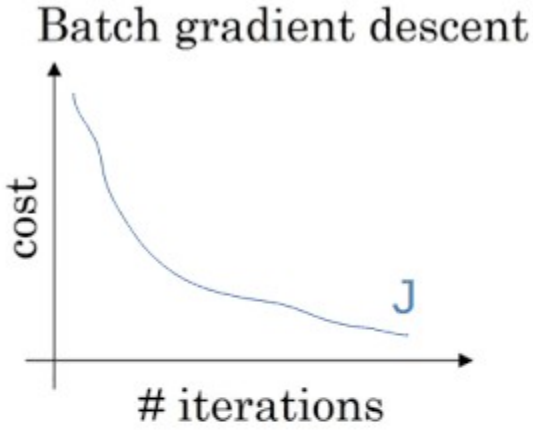
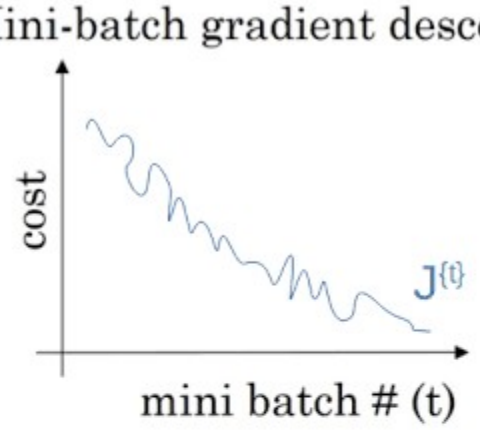
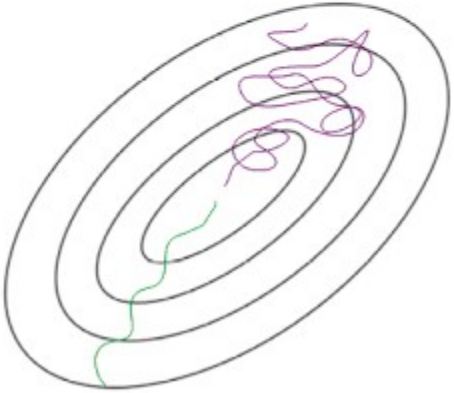


Figura 11-3. Função de ativação ELU

Mini-batch (“minilote”)







If mini-batch size = 1: Stocastic Gradient Descent = every example (row) it is used as mini-batch: lose sped-up from vectorization.

If mini-batch size = m: Batch Gradient Descent: $(X^{\{1\}}, Y^{\{1\}}) = (X, Y)$ = the entire training set is used: size = n: too long interations.

Normalização em minilote ("batch normalization")

A técnica consiste em adicionar uma operação no modelo imediatamente antes da função de ativação de cada camada, ao simplesmente centralizar em zero e normalizar as entradas, e então escalonar e deslocar o resultado utilizando dois novos parâmetros por camada (um para dimensionamento, outro para deslocamento). Em outras palavras, esta operação permite que o modelo aprenda a escala ideal e a média das entradas para cada camada.

Equação 11-3. Algoritmo de normalização em lote

1. $\mu_B = \frac{1}{m_B} \sum_{i=1}^{m_B} \mathbf{x}^{(i)}$

2. $\sigma_B{}^2 = \frac{1}{m_B} \sum_{i=1}^{m_B} \left(\mathbf{x}^{(i)} - \mu_B\right)^2$

3. $\widehat{\mathbf{x}}^{(i)} = \frac{\mathbf{x}^{(i)} - \mu_B}{\sqrt{\sigma_B{}^2 + \epsilon}}$

4. $\mathbf{z}^{(i)} = \gamma \widehat{\mathbf{x}}^{(i)} + \beta$

- m_B é o número de instâncias no minilote;
- $\widehat{\mathbf{x}}^{(i)}$ é a entrada normalizada e centralizada em zero;
- γ é o parâmetro de escala para a camada;
- β é o parâmetro de deslocamento (*offset*) para a camada;
- ϵ é um número minúsculo para evitar a divisão por zero (tipicamente 10^{-5}), chamado *smoothing term*;
- $\mathbf{z}^{(i)}$ é a saída da operação BN: é uma versão escalonada e deslocada das entradas.

- μ_B é a média empírica avaliada em todo o minilote B;
- σ_B é o desvio padrão empírico também avaliado em relação a todo o minilote;

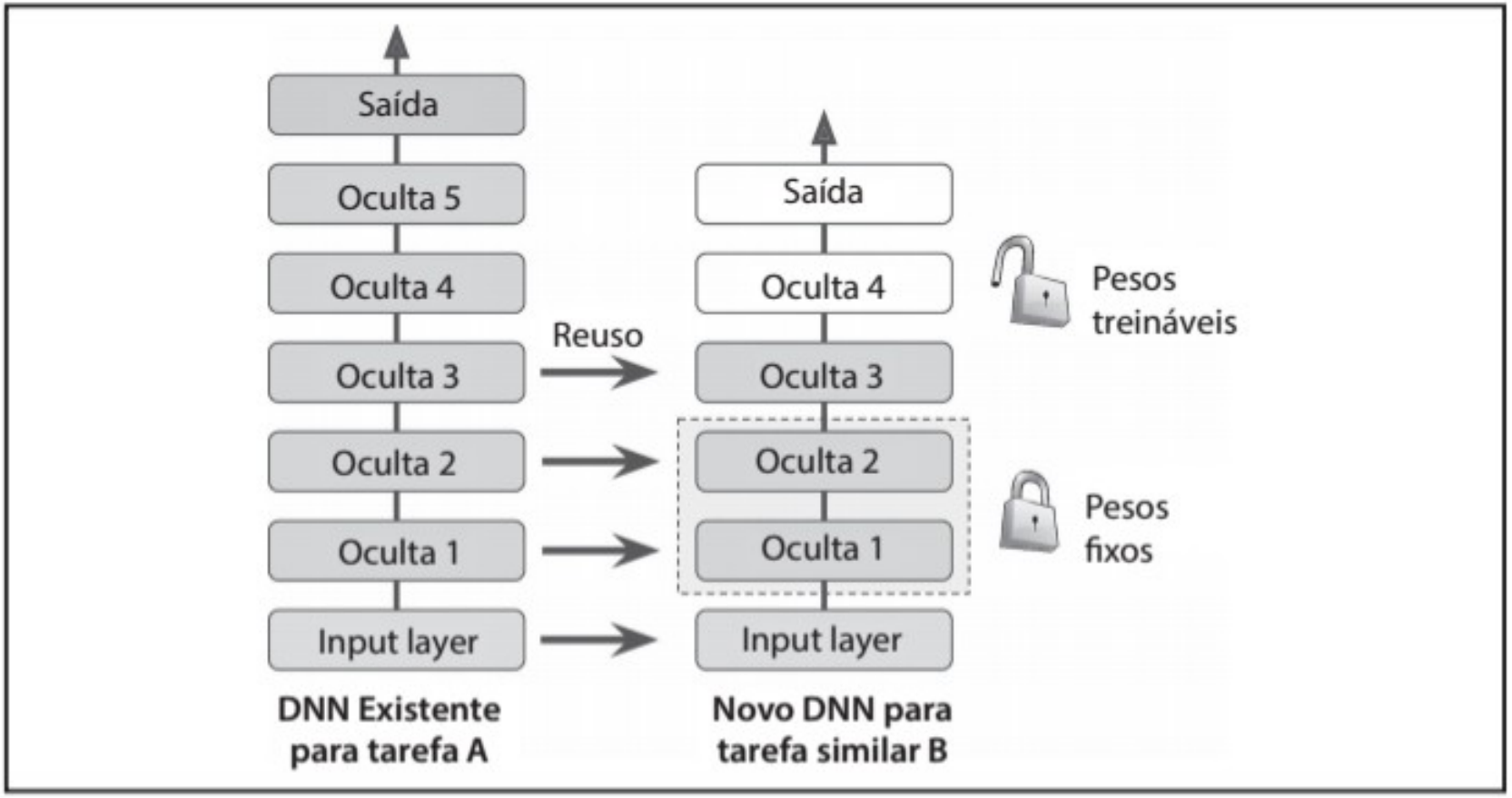
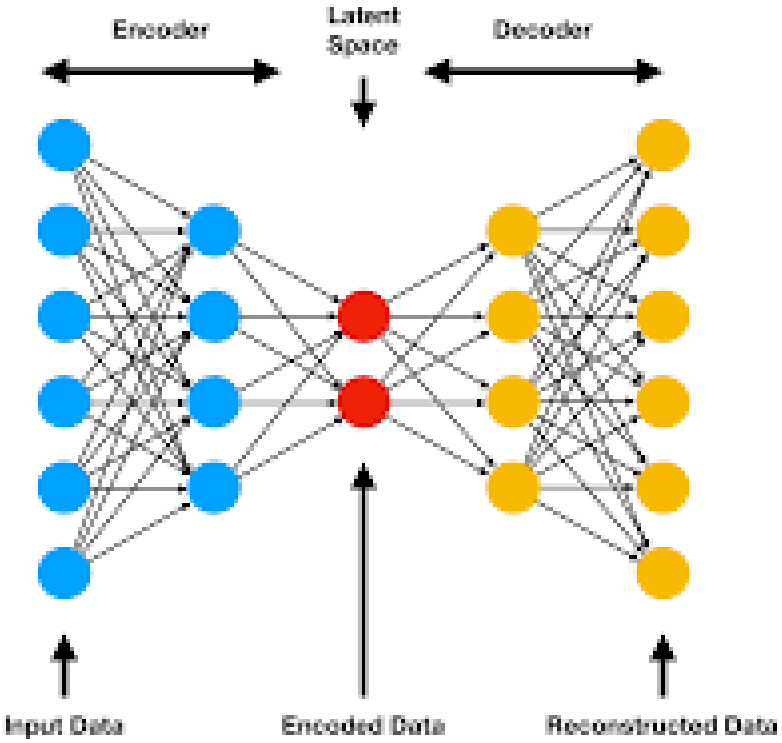


Figura 11-4. Reutilizando camadas pré-treinadas

Autoencoder



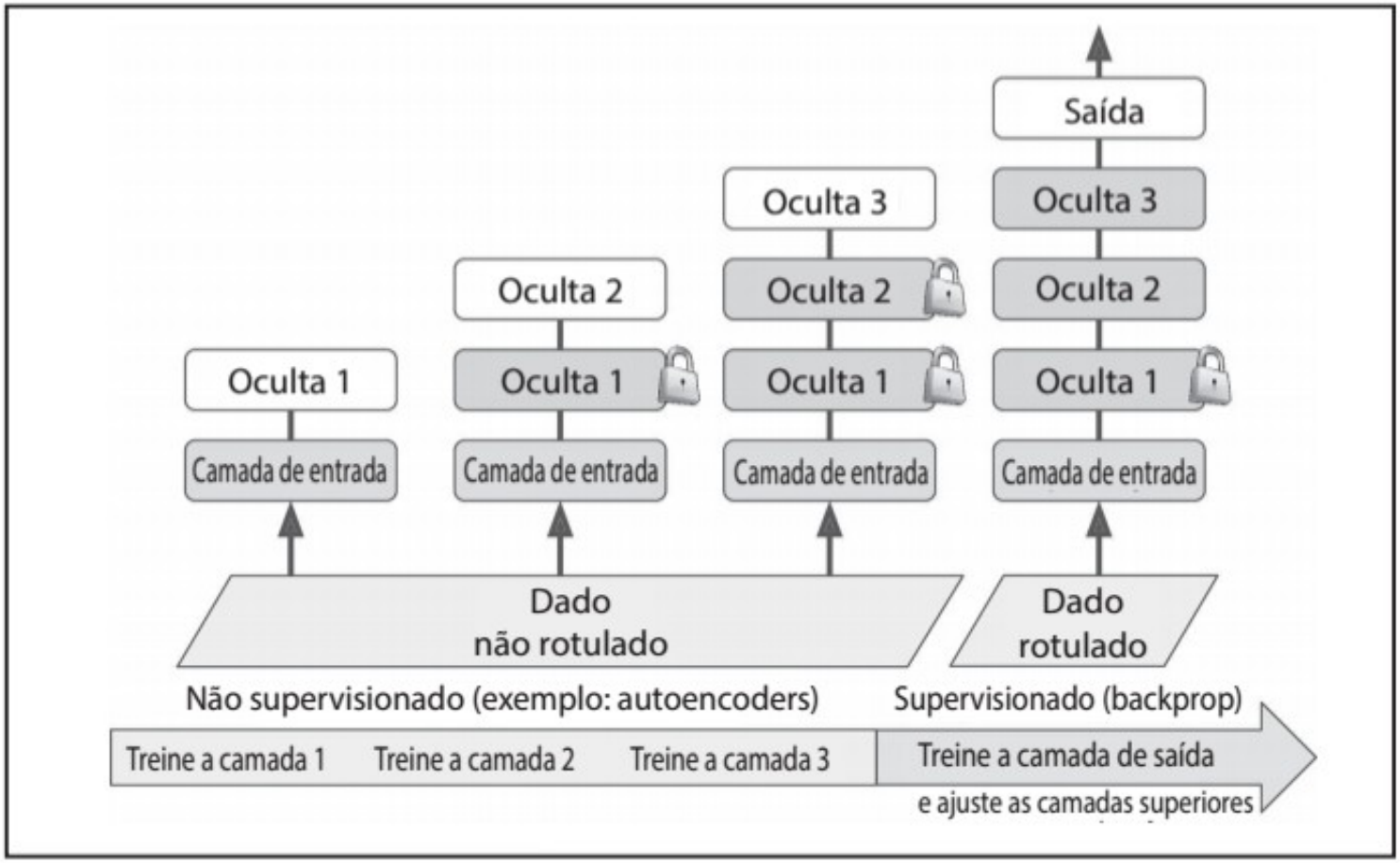


Figura 11-5. Pré-treinamento sem supervisão

Cronograma da taxa de aprendizado

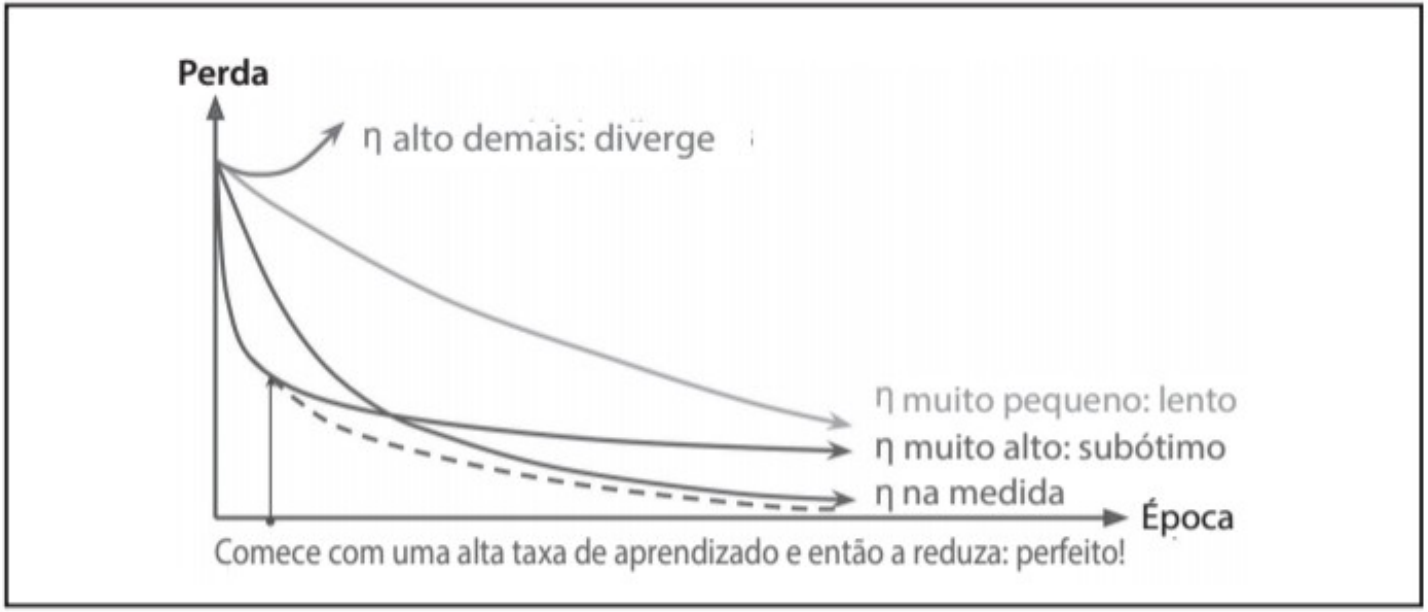


Figura 11-8. Curvas de aprendizado para várias taxas de aprendizado η

Regularização L1 e L2

L1 Regularization

$$\text{Cost} = \sum_{i=0}^{N} (y_i - \sum_{j=0}^{M} x_{ij} W_j)^2 + \lambda \sum_{j=0}^{M} |W_j|$$

L2 Regularization

$$\text{Cost} = \underbrace{\sum_{i=0}^{N} (y_i - \sum_{j=0}^{M} x_{ij} W_j)^2}_{\text{Loss function}} + \underbrace{\lambda \sum_{j=0}^{M} W_j^2}_{\text{Regularization Term}}$$

L1: $R(\theta) = \|\theta\|_1 = \sum_{i=1}^{n} |\theta_i|$

L2: $R(\theta) = \|\theta\|_2^2 = \sum_{i=1}^{n} \theta_i^2$

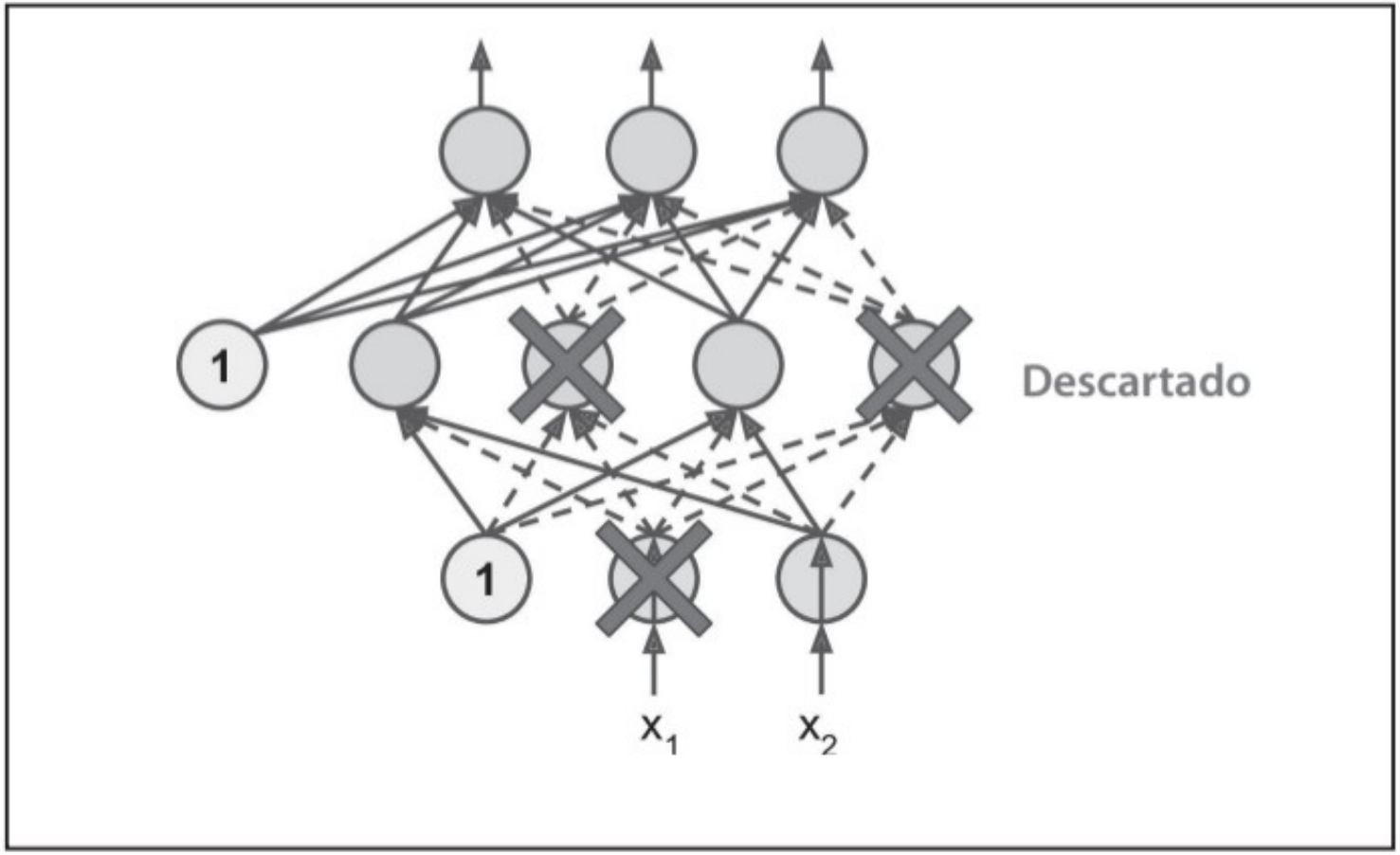


Figura 11-9. Regularização Dropout

Data augmentation: gerar novas instâncias para treino

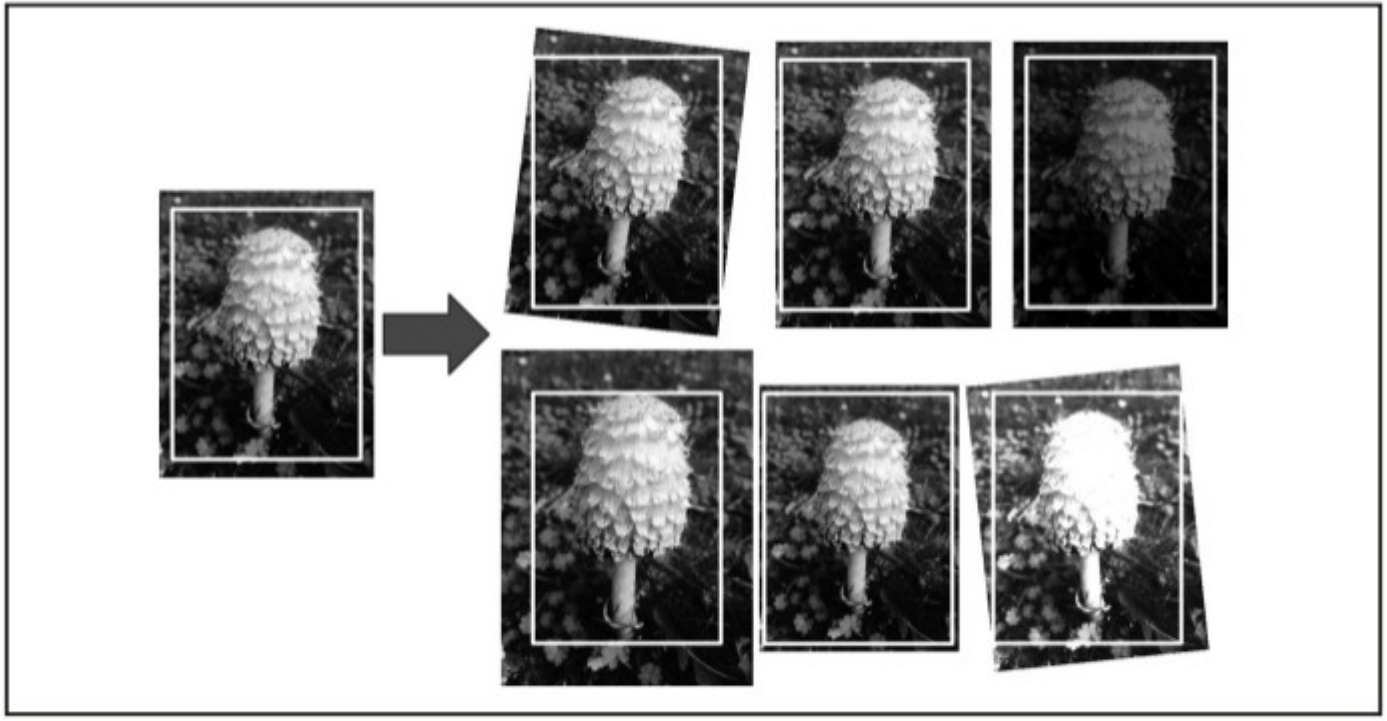


Figura 11-10. Gerando novas instâncias de treinamento a partir das já existentes

Sumário: dicas para uma 1a tentative com DNN

Tabela 11-2. Configuração Padrão da DNN

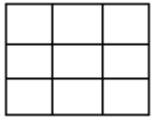
Inicialização	Inicialização He
Função de Ativação	ELU
Normalização	Normalização em Lote
Regularização	Dropout
Otimizador	Gradiente Acelerado de Nesterov
Cronograma da taxa de aprendizado	Nenhum

O que são redes convolucionais (CNN)?

Usam o princípio da convolução: $y(t) = x(t) * h(t)$, onde $h(t)$ é a resposta ao impulso, chamada de filtro, máscara ou kernel em proc. de imagens ou “feature” em CNN

Exemplos:

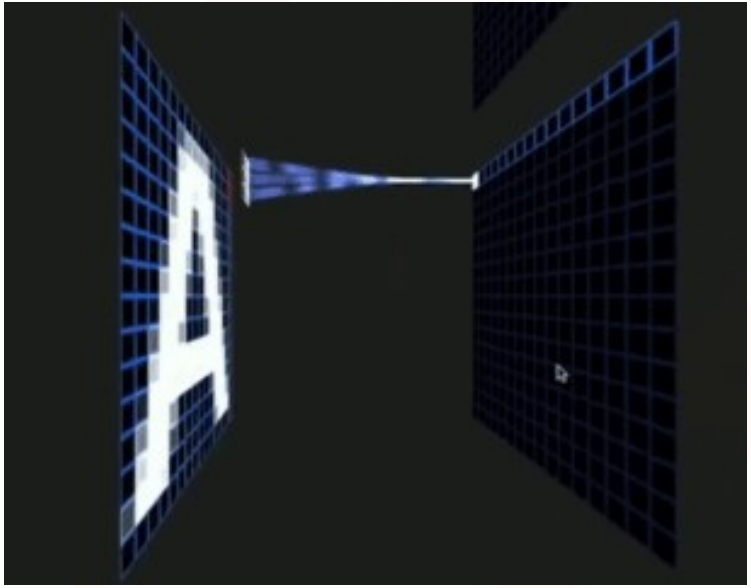
Dimensão de kernel: 3 x 3 pixels



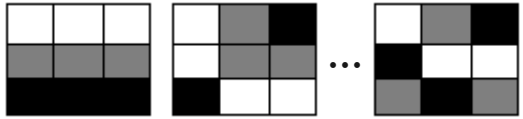
Filtro da animação

-1	0	1
-1	0	1
-1	0	1

representado em cor



Outros filtros:



A CNN aprende bons filtros para a dada tarefa

CNN: cortex visual

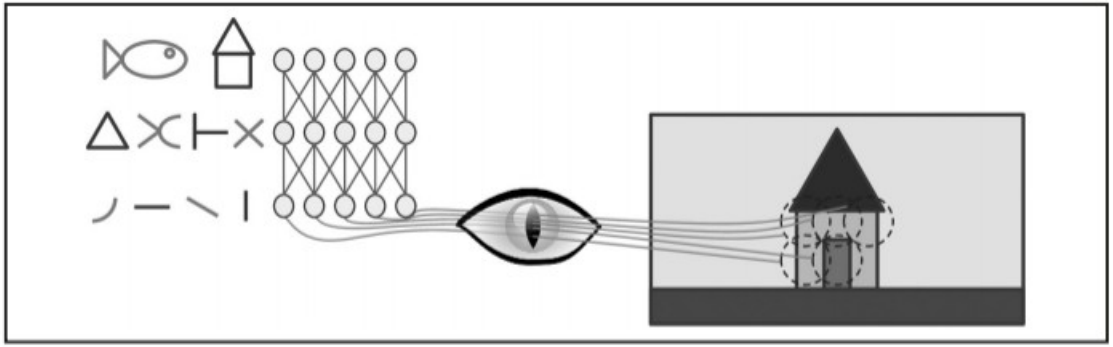
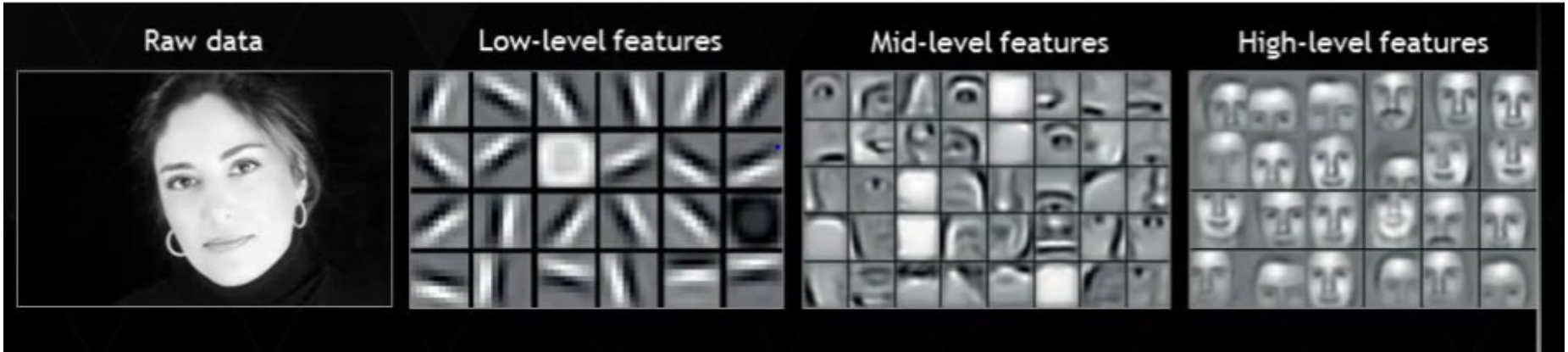
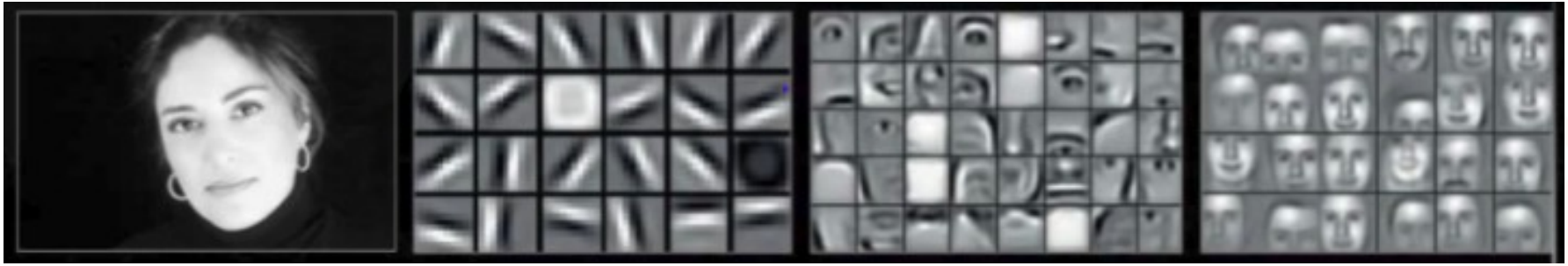


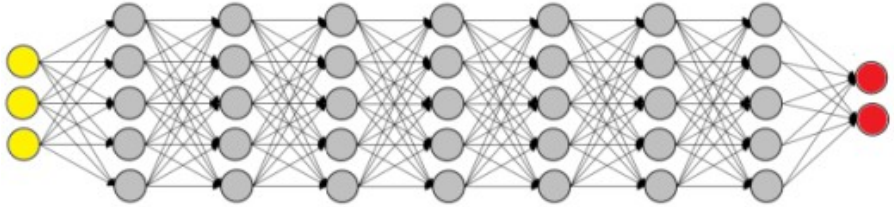
Figura 13-1. Campos receptivos locais no córtex visual



Redes neurais profundas convolucionais (CNNs) aprendem padrões de forma hierárquica

Detecção de rostos: primeiras camadas identificam padrões simples, e últimas camadas os mais complexos

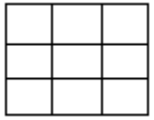




Processamento em camada convolucional de CNN

Exemplos:

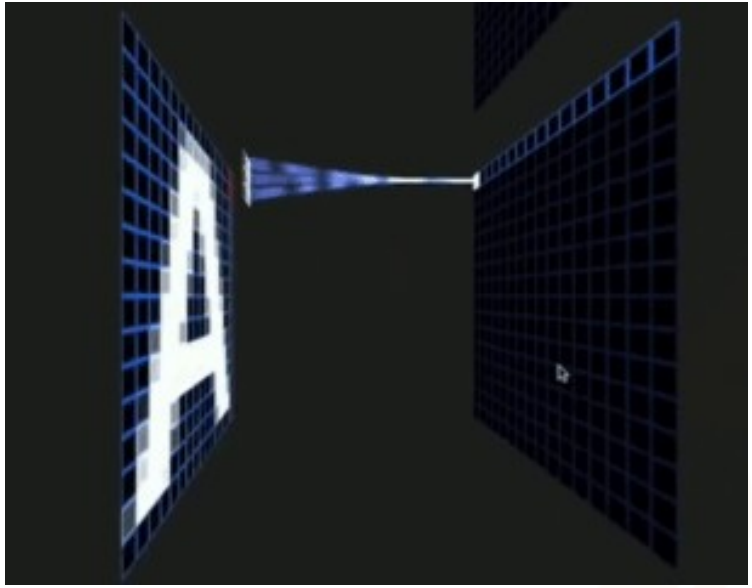
Dimensão do filtro:
3 x 3 pixels



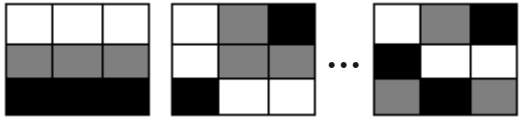
Filtro da animação

-1	0	1
-1	0	1
-1	0	1

representado em cor



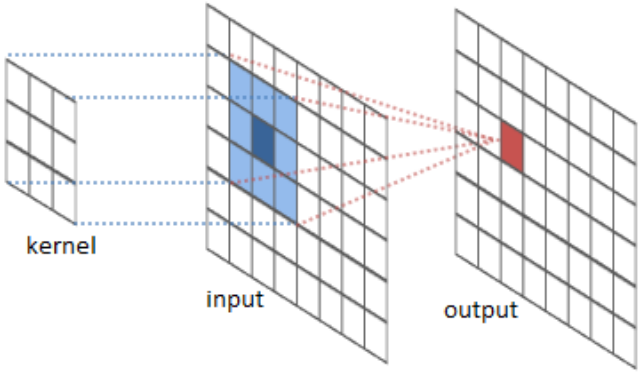
Outros filtros:

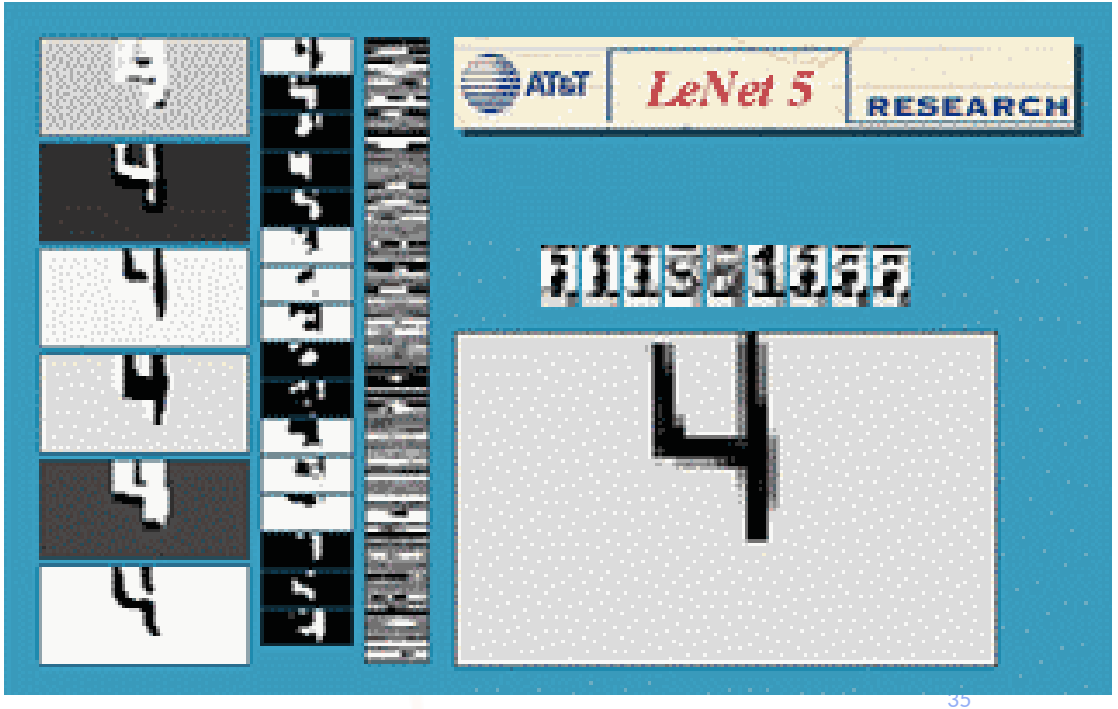


A CNN aprende bons filtros para a tarefa dada

LeNet 5 CNN

Approximation involves **locality** at different scales and shift invariance





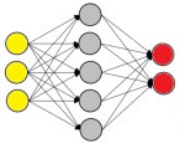
Objetivo: aprender nomenclatura

Tabela 13-1. Arquitetura LeNet-5

Camada	Tipo	Mapas	Tamanho	Tamanho do Kernel	Stride	Ativação
Out	Totalmente conectado	–	10	–	–	RBF
F6	Totalmente conectado	–	84	–	–	tanh
C5	Convolução	120	1×1	5×5	1	tanh
S4	Média de Pooling	16	5×5	2×2	2	tanh
C3	Convolução	16	10×10	5×5	1	tanh
S2	Média de Pooling	6	14×14	2×2	2	tanh
C1	Convolução	6	28×28	5×5	1	tanh
In	Entrada	1	32×32	–	–	–

Marcos históricos em NN e visão computacional

MLP com arquitetura "densa"

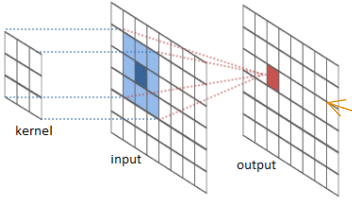


[Rosenblatt] "Perceptron"

[Rumelhart et al] "Multilayer Perceptron" (backprop)

[Schmidhuber et al] LSTM, recorrente

1957 | 1980 | 1986 | 1989 | 1994 | 2006 | 2011 | 2012



[Fukushima] Camada "convolucional"

[LeCun et al] CEP com convnet LeNet5, 3 camadas, 9760 parâmetros

[Hinton et al] Deep semi-supervised belief nets

[Krizhevsky et al] venceu competição ImageNet, 8 camadas, 61M parâmetros

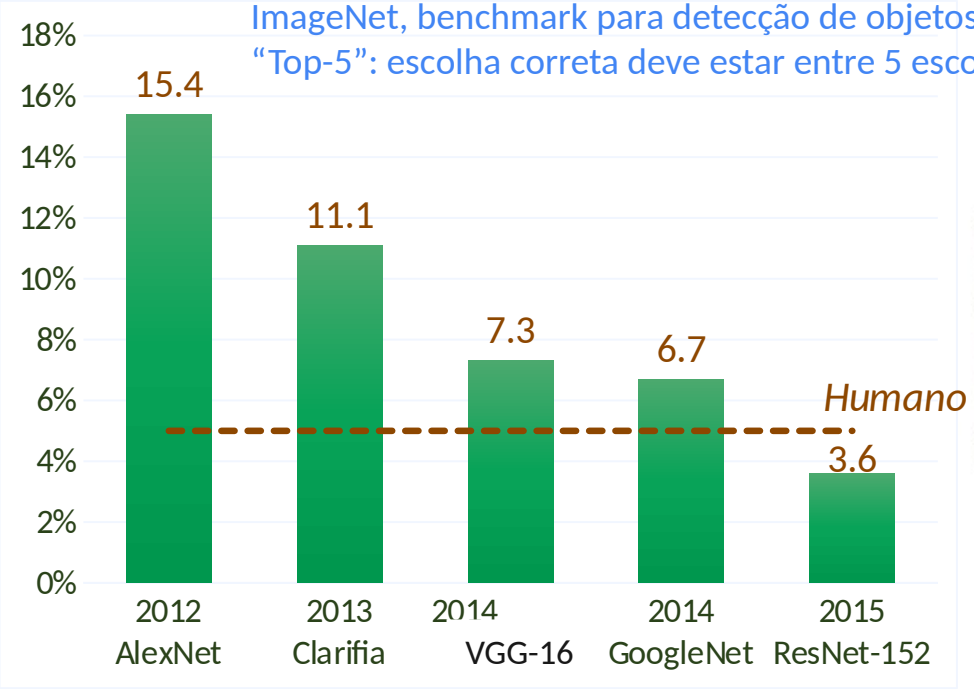
Algoritmos

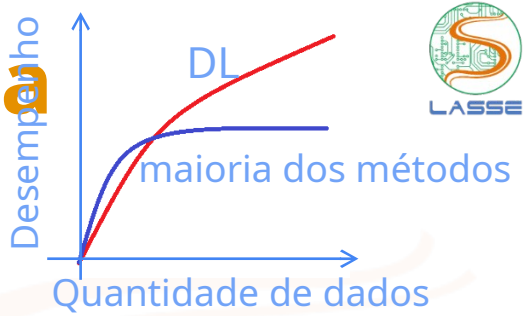
Visão computacional

Lições de visão computacional

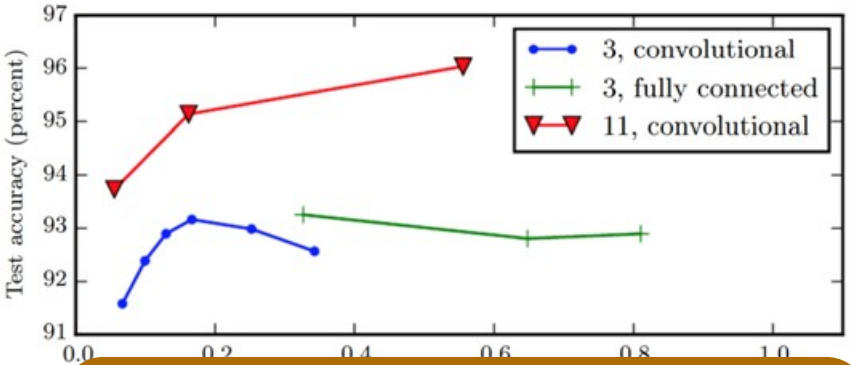
26.2 (2º lugar em 2012)

Breakthrough:
ImageNet, benchmark para detecção de objetos
"Top-5": escolha correta deve estar entre 5 escolhas





L1 – Desempenho escala com dados



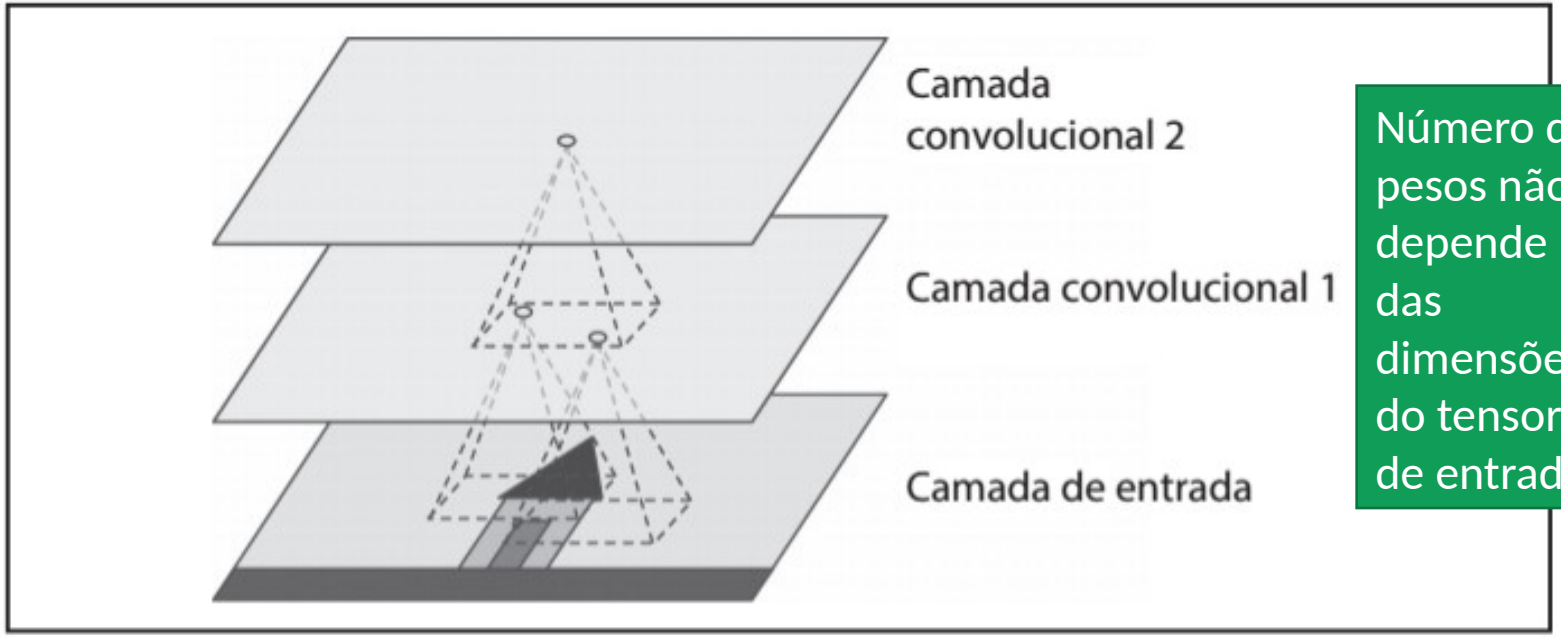
L2 – Não requer "feature engineering":
convnet aprende representações internas

A State-of-the-Art Survey on Deep Learning Theory and Architectures, Zahangir Alom et al, 2019.

CNN: parcialmente conectada

Por que não simplesmente utilizar uma rede neural profunda regular, com camadas totalmente conectadas, para tarefas de reconhecimento de imagem? Infelizmente, embora isso funcione bem para pequenas imagens (por exemplo, MNIST), falha em imagens maiores devido ao grande número de parâmetros necessários. Por exemplo, uma imagem 100 × 100 tem 10 mil pixels e, se a primeira camada tiver apenas 1 mil neurônios (o que já restringe severamente a quantidade de informações transmitidas para a próxima camada), isso significa um total de 10 milhões de conexões. E essa é apenas a primeira camada. As CNNs resolvem este problema ao utilizar camadas parcialmente conectadas.

Camada convolucional: pesos são os mesmos à medida que kernel é transladado ao longo do tensor de entrada



Número de pesos não depende das dimensões do tensor de entrada!

Figura 13-2. Camadas CNN com campos receptivos locais retangulares

Fatos da convolução linear

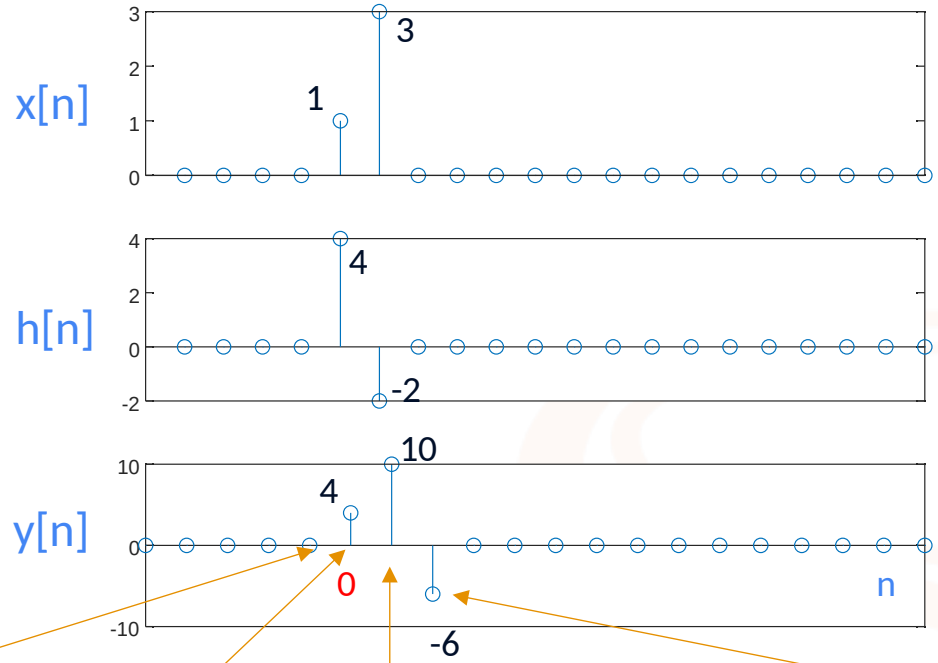
- a convolução 1D completa entre sequências x e h com N_x e N_h amostras cada uma, respectivamente, tem duração $N_x + N_h - 1$. Ou seja, se temos a entrada x com $N_x = 10$ amostras e convoluímos com h tendo $N_h = 3$ amostras, a saída da convolução completa teria 12 amostras, ou seja, **mais longa** até do que a entrada x.

- Tal convolução completa não é usada em redes convolucionais pois as amostras de saída nos extremos (bordas) estariam sendo calculadas com poucas amostras de x e h.

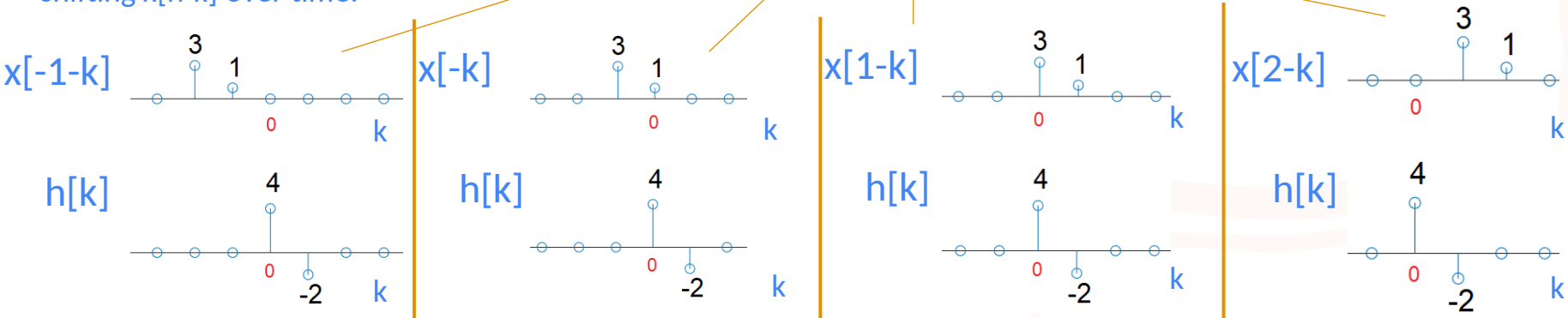
Convolução 1-D

$$y[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k]$$

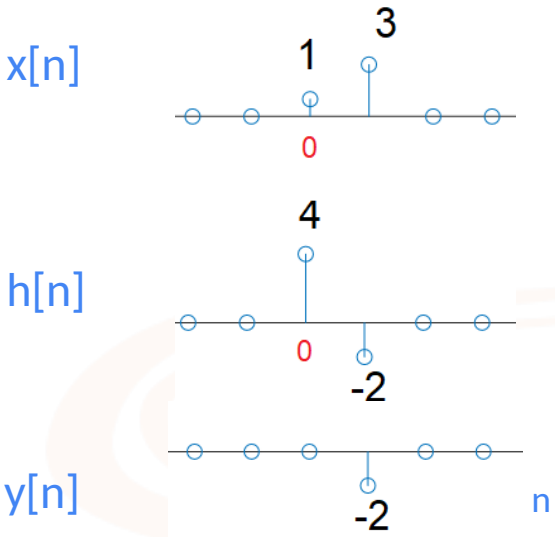
```
x=[1 3];
h=[4 -2];
y=conv(x,h)

Result:
y=[4  10  -6]
```



Flipping x over the y-axis and shifting x[n-k] over time:



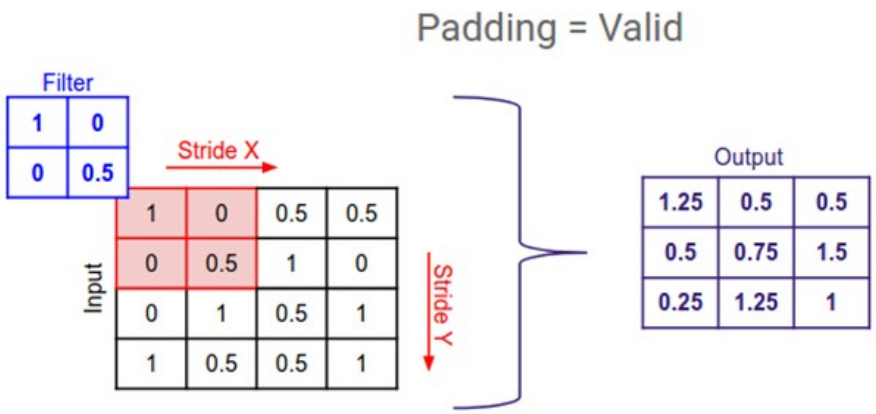
“Convolução” em proc. de imagens e deep learning



- Não há “flipping”. De fato implementa-se uma “correlation”
- A resposta ao impulso é chamada “kernel”
- Na opção “padding=valid”, apenas se calcula a “saída” quando há superposição completa do kernel h[n] com a entrada x[n]
- O valor de n não é necessariamente incrementado de 1, mas sim desclocado por um valor chamado “stride”
- Na opção “valid”, o resultado da “convolução” y[n] terá uma dimensão menor do que a entrada x[n]
- Para que y[n] e x[n] tenham a mesma dimensão, usa-se a opção padding = ‘same’, que preenche os extremos de x[n] com valores zero (“zero padding”)

Convolução é multidimensional: o mesmo filtro ("kernel") é aplicado a todos tensores ao longo da dimensão "depth" (profundidade)

- Exemplo com imagens e "depth = 1"



Example

0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

Kernel

0	-1	0
-1	5	-1
0	-1	0

114				

Source code: https://stackoverflow.com/questions/52067833/how-to-plot-an-animated-matrix-in-matplotlib

Padding (“preenchimento”)

- As opções para lidar com as bordas mais comuns em redes neurais são duas:

1) “gerar imagem na saída com a mesma dimensão da entrada” (ou seja, apenas o “depth” do tensor de entrada é alterado. Para isso, usa-se “zero padding” (adiciona-se tantas linhas e colunas às bordas da imagem de entrada quanto necessárias). No Keras, isso é especificado com padding = ‘same’ (pois a imagem de saída tem a mesma (“same”) dimensão da entrada se o stride=1)
2) “gerar imagem na saída com dimensão menor do que da entrada”, pois exige-se que para cada convolução o kernel tenha sobreposição total com a imagem de entrada. No Keras, isso é especificado com padding = ‘valid’ (pois toda convolução usou uma região da entrada que tinha todos os pixels válidos)

Stride and padding

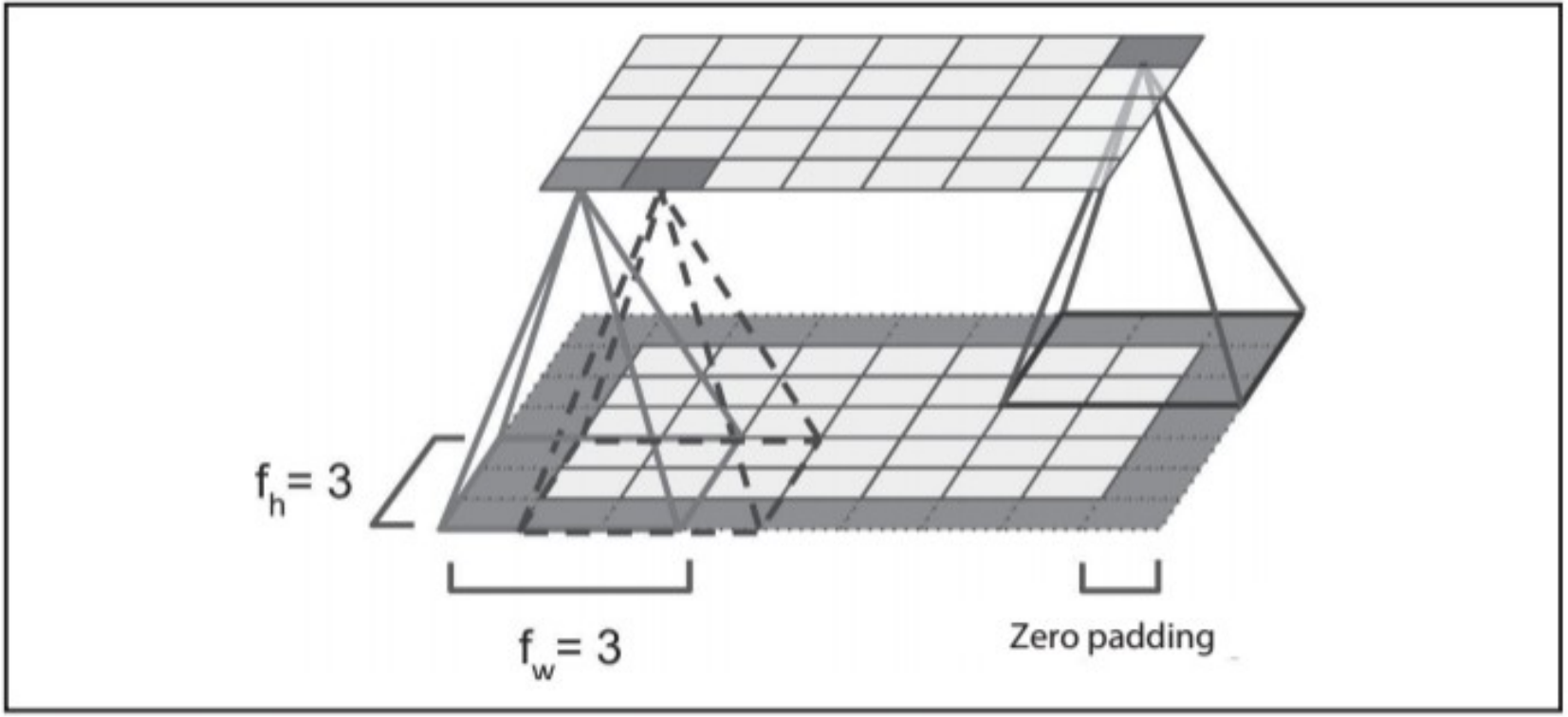


Figura 13-3. Conexões entre camadas e zero padding

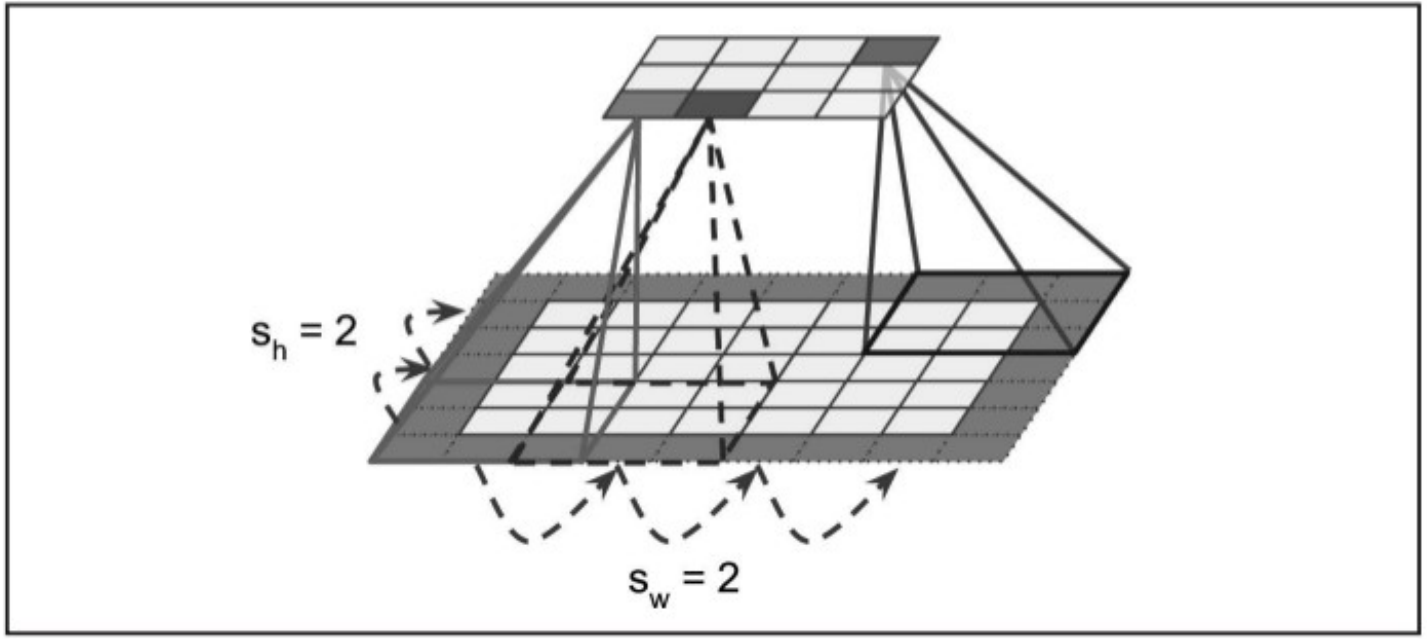


Figura 13-4. Reduzindo a dimensionalidade utilizando um stride de 2

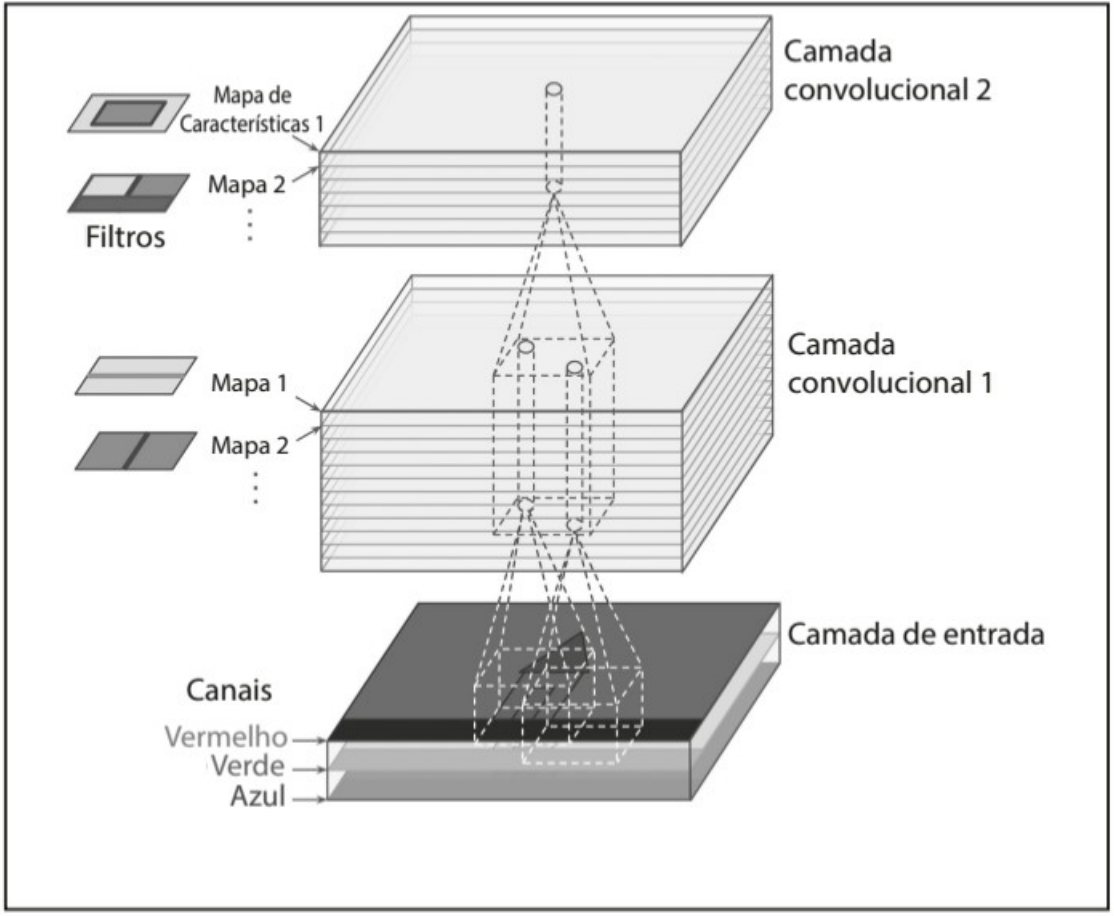


Figura 13-6. Camadas de convolução com vários mapas de características e imagens com três canais

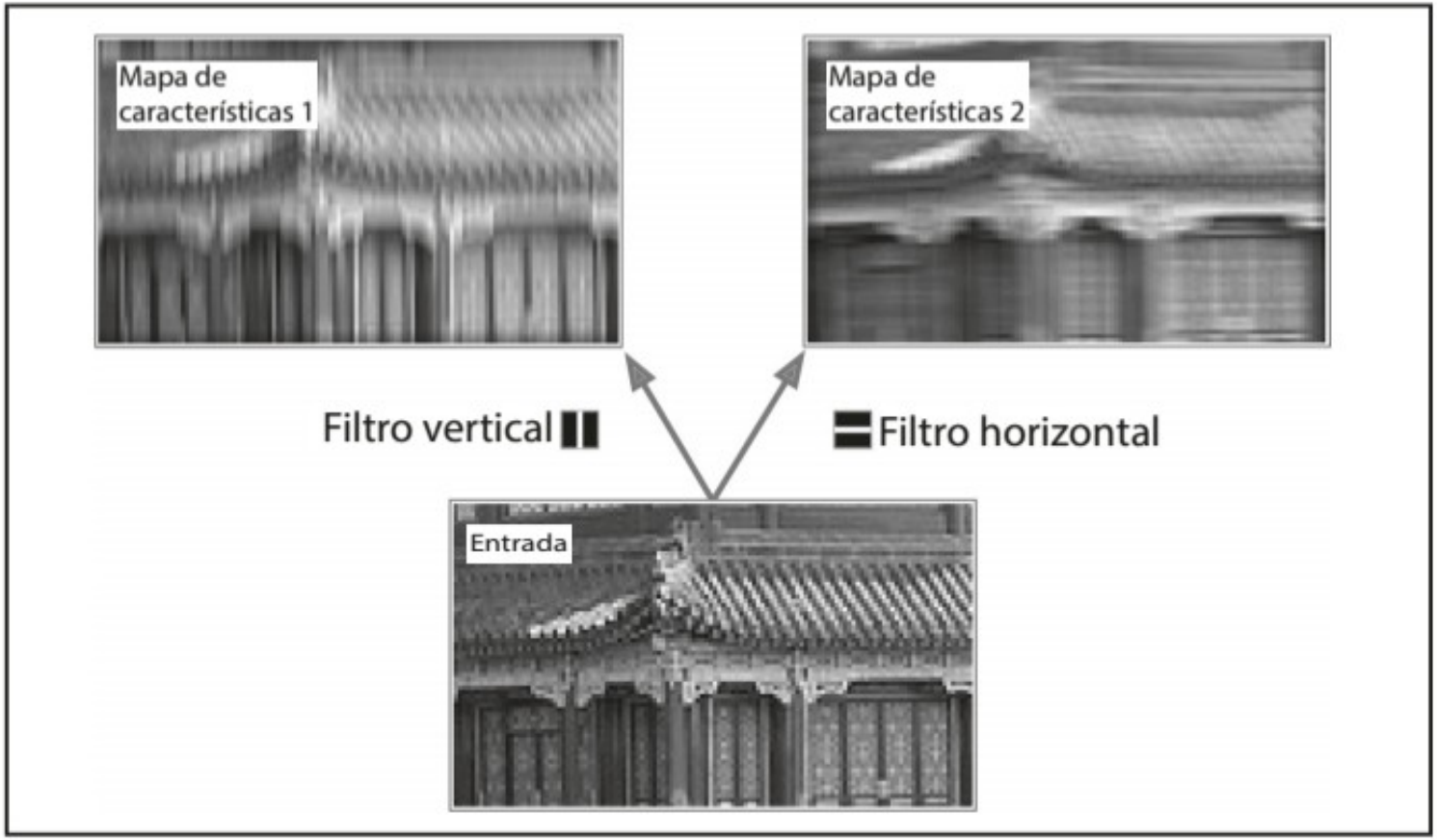


Figura 13-5. Aplicando dois filtros diferentes para obter dois mapas de características

Convolução multidimensional

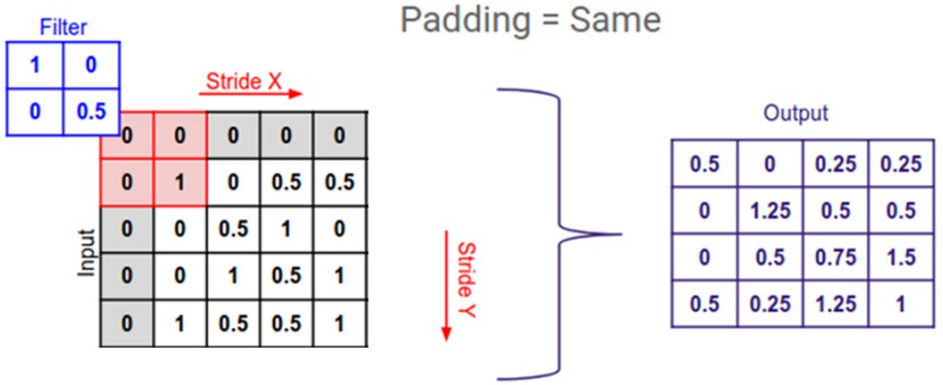
Equação 13-1. Calculando a saída de um neurônio em uma camada convolucional

$$z_{i,j,k} = b_k + \sum_{u=0}^{f_h-1} \sum_{v=0}^{f_w-1} \sum_{k'=0}^{f_{n'}-1} x_{i',j',k'} \cdot w_{u,v,k',k} \quad \text{com} \begin{cases} i' = i \times s_h + u \\ j' = j \times s_w + v \end{cases}$$

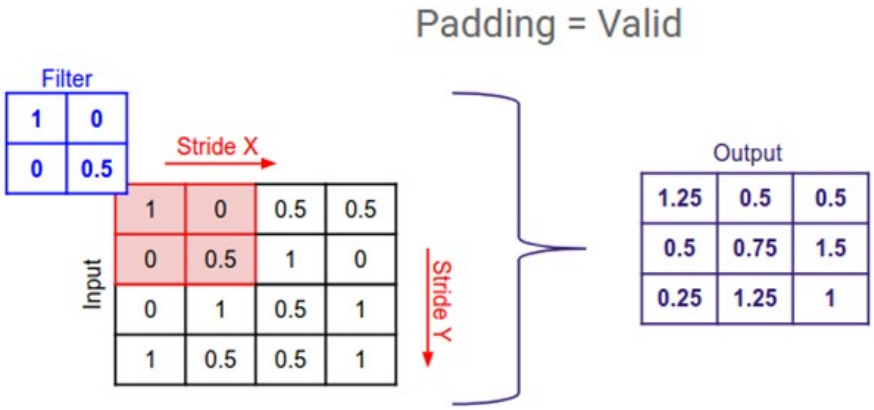
- $z_{i,j,k}$ é a saída do neurônio localizado na linha i, coluna j no mapa de características k da camada convolucional (camada l);
- Como explicado anteriormente, s_h e s_w são os strides verticais e horizontais, f_h e f_w são a altura e a largura do campo receptivo e $f_{n'}$ é o número de mapas de características na camada anterior (camada $l - 1$);
- $x_{i',j',k'}$ é a saída do neurônio localizado na camada $l - 1$, linha i', coluna j', mapa de características k' (ou canal k' se a camada anterior for a camada de entrada);
- b_k é o termo de polarização para o mapa de características k (na camada l). Você pode encarar isso como um botão que altera o brilho geral do mapa de características k;
- $w_{u,v,k',k}$ é o peso da conexão entre qualquer neurônio no mapa de características k da camada l e sua entrada localizada na linha u, coluna v (relativa ao campo receptivo dos neurônios) e ao mapa de características k'.

Padding = Same

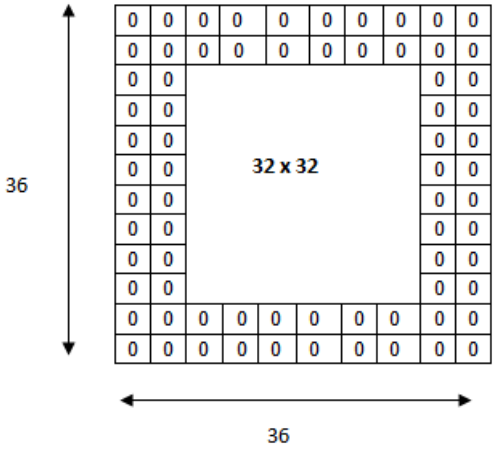
- O número de linhas e colunas no “zero-padding” é feito de acordo com a dimensão do kernel. No exemplo da figura anterior, como o kernel é 2x2, é necessário acrescentar uma linha e uma coluna de zeros. Se o kernel fosse 7x7, daí 6 linhas e 6 colunas de zeros seriam acrescidas, repartindo-se 3 para cada lado e acima/abaixo, de forma que a imagem continue centralizada.
  - Um detalhe: No Tensorflow e Keras, quando a dimensão do kernel é par, a coluna e/ou linha extra de zeros é colocada à direita e/ou abaixo da imagem, e não à esquerda e acima como na figura anterior.



- Se o stride=1 e padding=same, a dimensão da imagem de saída é a mesma da entrada (excetuando o depth). Nota-se também que se os strides nas direções X e Y, ou a dimensão do kernel em X e Y não forem o mesmo, deve-se calcular para cada dimensão.



- Como informado, normalmente as dimensões do kernel são números ímpares. Digamos que o kernel é 3 x 3. Daí no “zero padding” se deve inserir 1 linha extra com zeros, acima e abaixo da imagem, além de 1 coluna de zeros à esquerda e à direita, e manter a imagem centralizada. Para um kernel 5 x 5, são duas ao invés de uma, como mostrado abaixo para uma matriz originalmente de 32 x 32 pixels:



Usar padding = “SAME” mas com stride > 1

- Este caso não gera saída com mesma dimensão da entrada!

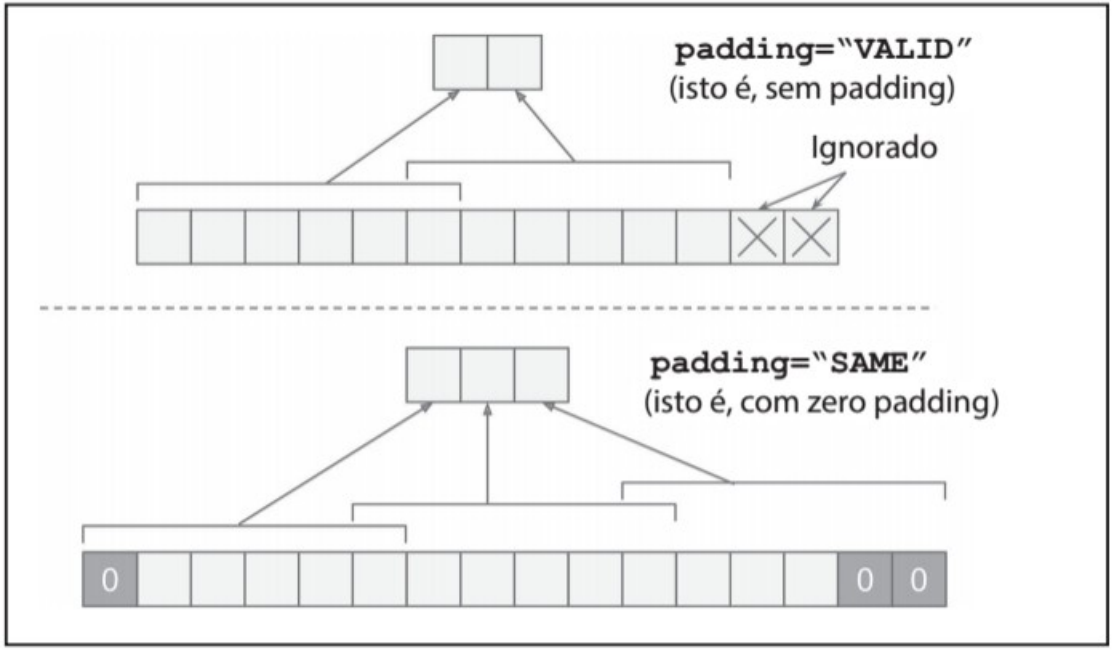


Figura 13-7. Opções de padding — largura da entrada: 13, largura do filtro: 6, stride: 5

Pooling

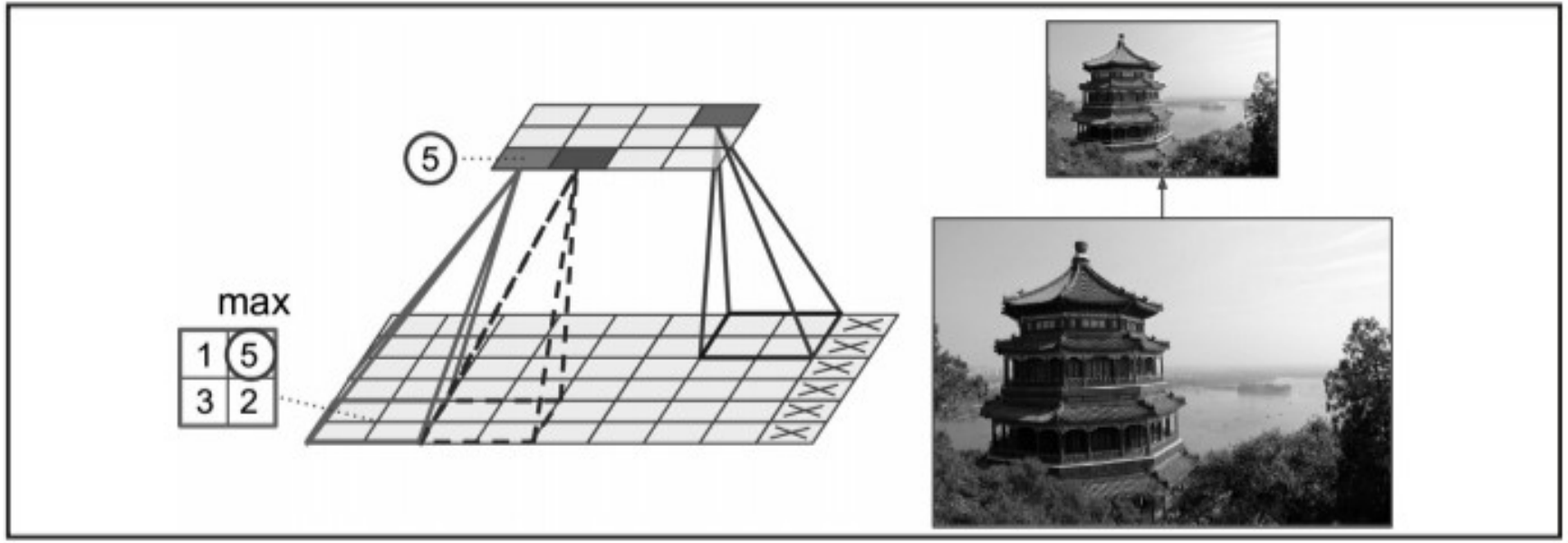
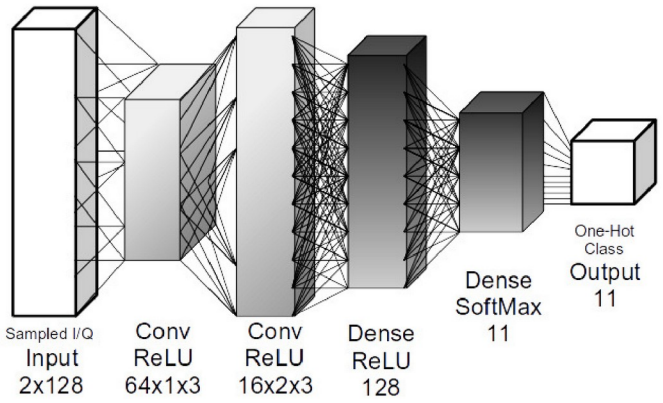


Figura 13-8. Camada max pooling (kernel pooling 2 × 2, stride de 2, sem padding)

Example of notation used for CNNs

- O'Shea



El Gamal

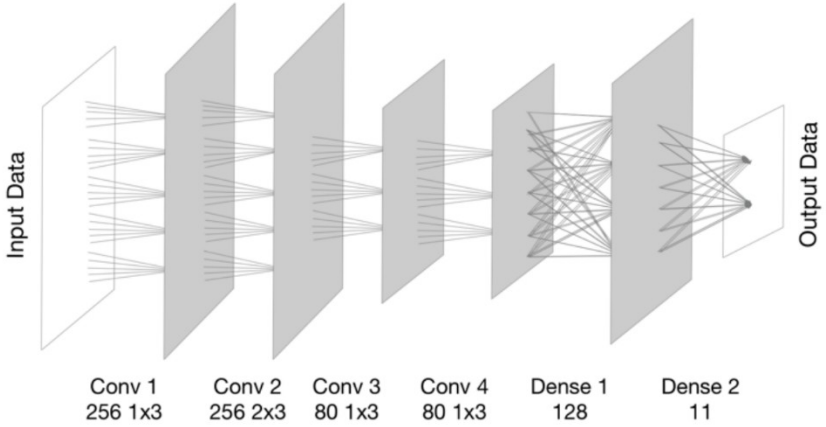


Fig. 2: Architecture of seven-layer CNN

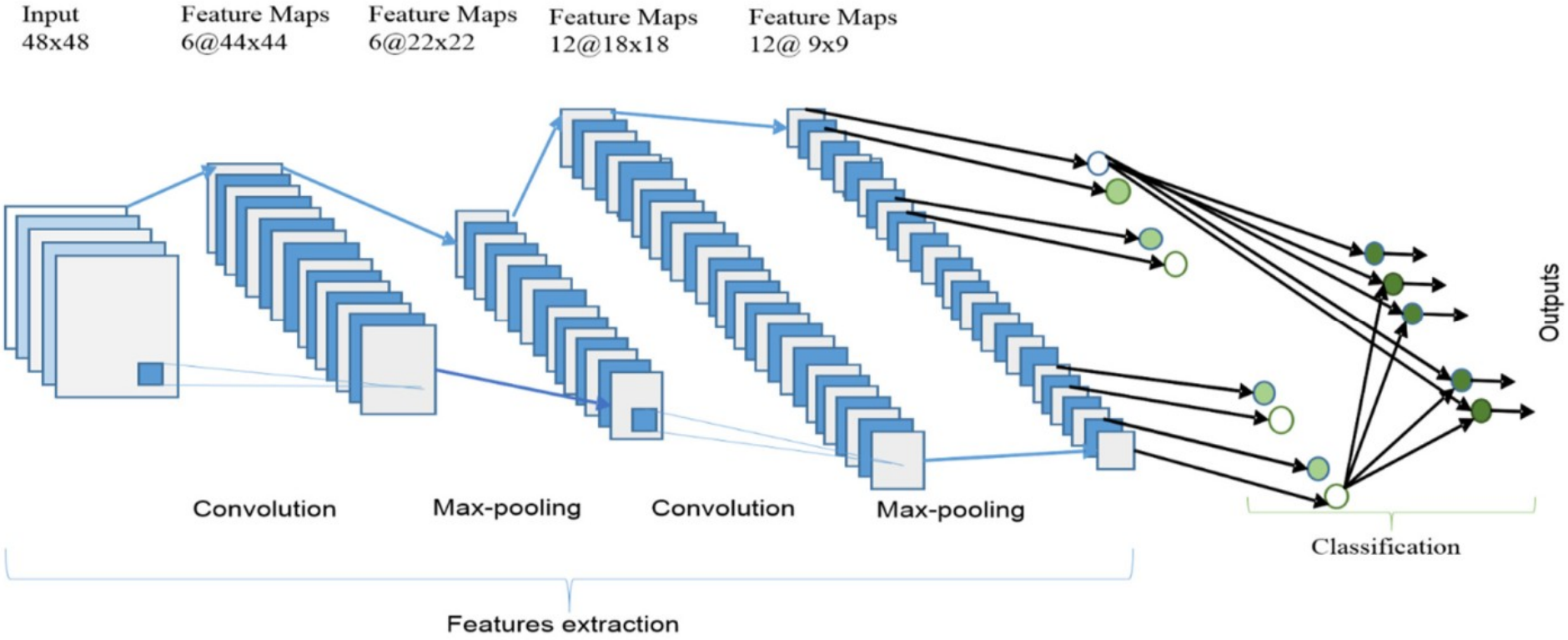


Figure 9. The overall architecture of the Convolutional Neural Network (CNN) includes an input layer, multiple alternating convolution and max-pooling layers, one fully-connected layer and one classification layer.

Outras redes

ResNet combats vanishing gradient

- Residual Network (or ResNet): winner of the ILSVRC 2015 challenge (http://www.image-net.org/challenges/LSVRC/)
- Developed by Kaiming He et al., extremely deep CNN composed of 152 layers. Key to train such deep network: skip (shortcut) connections
- The signal feeding into a layer is also added to the output of a layer located a bit higher up the stack

Residual network

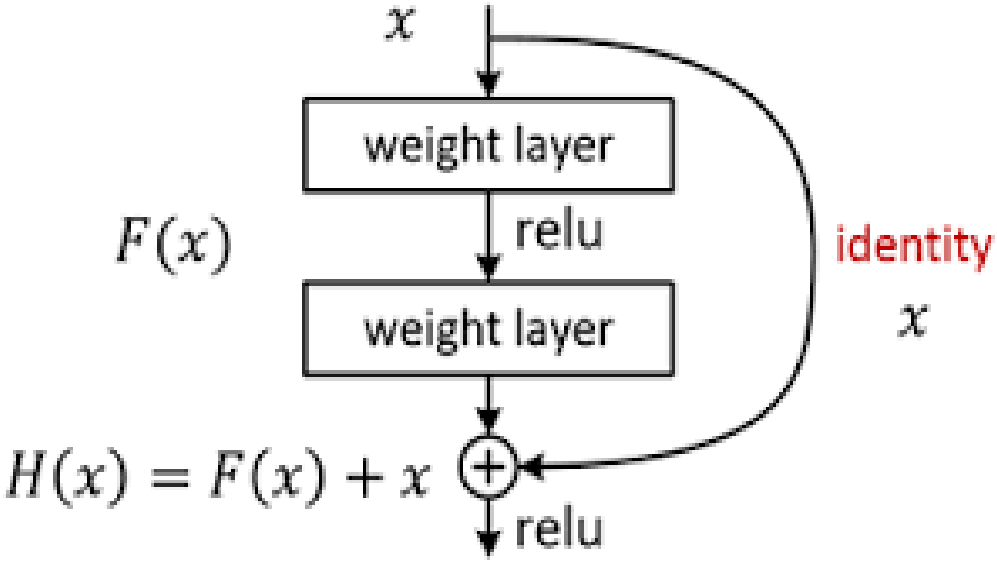


Figure 2 Residual Network (Zhang, 2015)

ResNet

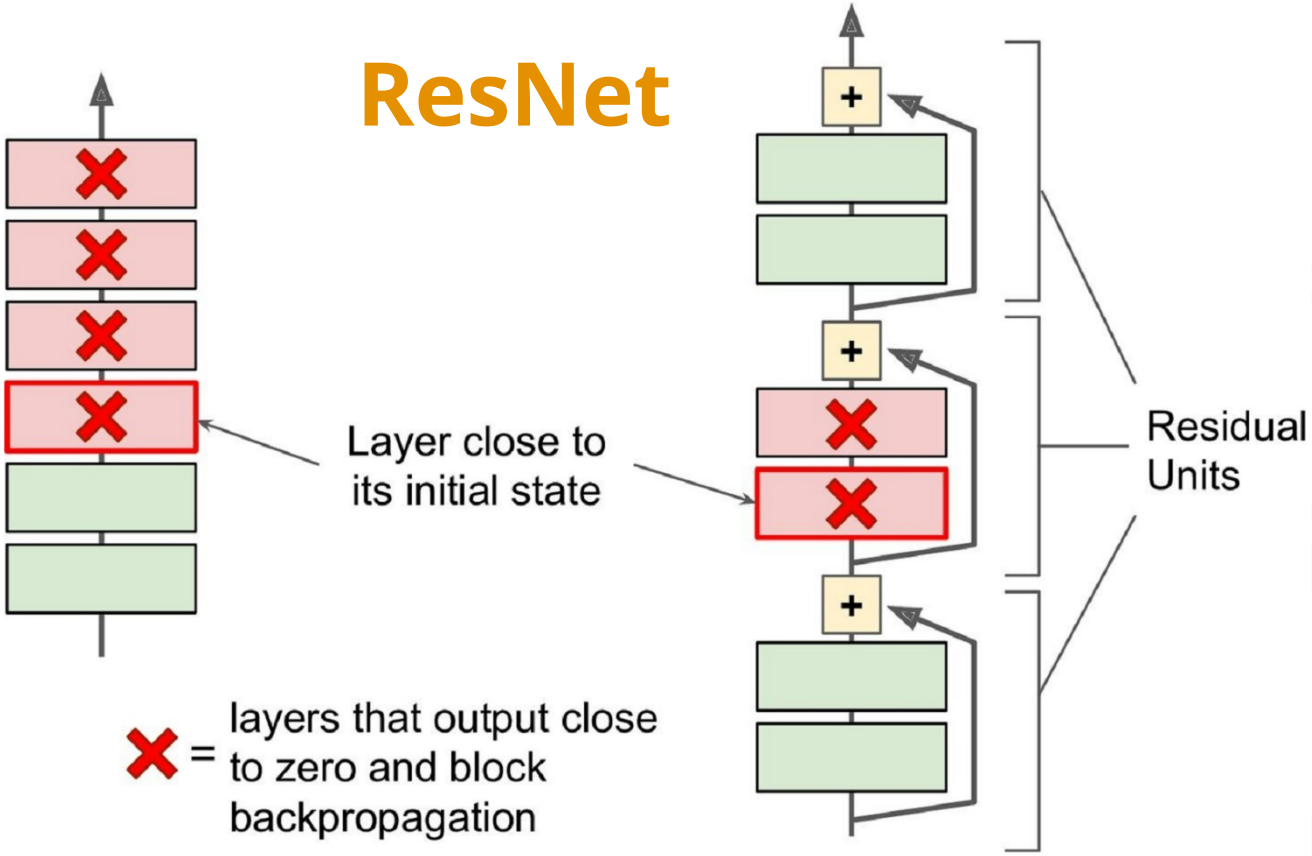


Figure 13-13. Regular deep neural network (left) and deep residual network (right)

From Gerón's book